

```
1 var _runtime = require('./runtime.js');
2 var _nativeApi = _runtime.getNativePlatformApi();
3 var _diagMsgs = [];
4 var _diagStart = Date.now();
5 function _diag(msg) {
6   var ts = Date.now() - _diagStart;
7   var line = '[' + ts + 'ms]' + msg;
8   _diagMsgs.push(line);
9   try { console.log(line); } catch (_) {}
10 }
11 function _showDiag() {
12   try {
13     if (_nativeApi && _nativeApi.showModal) {
14       _nativeApi.showModal({
15         title: '启动诊断',
16         content: _diagMsgs.join('\n'),
17         showCancel: false,
18       });
19     }
20   } catch (_) {}
21 }
22 _diag('game.js 开始执行, host=' + _runtime.detectMinigamePlatform());
23 try {
24   if (_nativeApi) {
25     var _si = _nativeApi.getSystemInfoSync();
26     _diag('platform:' + _si.platform + ' system:' + _si.system);
27   }
28 } catch (e) {
29   _diag('getSystemInfo失败:' + e);
30 }
31 try {
32   if (typeof GameGlobal !== 'undefined') {
33     GameGlobal.onError = function (msg) {
34       _diag('onError:' + msg);
35       _showDiag();
36     };
37     GameGlobal.onUnhandledRejection = function (ev) {
38       _diag('unhandledRej:' + (ev && ev.reason || ev));
39       _showDiag();
40     };
41   }
42 } catch (_) {}
43 _diag('加载 pixi-adapter...');
44 try {
45   require('./pixi-adapter/index');
46   _diag('pixi-adapter OK');
47 } catch (e) {
48   _diag('pixi-adapter 失败!!!' + e);
49   _showDiag();
50 }
```

```
51 if (typeof Intl === 'undefined') {
52   _diag('Intl不存在,注入polyfill');
53   var _g = typeof GameGlobal !== 'undefined' ? GameGlobal : (typeof globalThis !== 'undefined' ? ...
54   _g.Intl = {});
55 }
56 var _started = false;
57 function _startGame(reason) {
58   if (_started) return;
59   _started = true;
60   if (reason) _diag(reason);
61   _diag('加载 game-bundle...');
62   try {
63     require('./game-bundle.js');
64     _diag('game-bundle OK');
65   } catch (e) {
66     _diag('game-bundle 失败!!!' + e);
67     _showDiag();
68   }
69   _diag('全部加载完成');
70 }
71 function _errText(err) {
72   return String((err && (err.errMsg || err.message)) || err || '');
73 }
74 function _isDevtools() {
75   try {
76     return !!( _nativeApi && _nativeApi.getSystemInfoSync && _nativeApi.getSystemInfoSync().platf...
77   } catch (_) {
78     return false;
79   }
80 }
81 function _requireImagesEntry() {
82   try {
83     require('./subpkg_images/game.js');
84     return true;
85   } catch (_) {
86     return false;
87   }
88 }
89 function _requireKitchenEntry() {
90   try {
91     require('./subpkg_kitchen/game.js');
92     return true;
93   } catch (_) {
94     return false;
95   }
96 }
97 function _loadImagesThenStart() {
98   // 开发者工具里 loadSubpackage 常挂起或不回调, 本地文件已在磁盘, 直接开游戏。
99   if (_isDevtools()) {
100     if (_requireImagesEntry()) _diag('devtools 已 require 图片分包入口');
```

```
101 if (_requireKitchenEntry()) _diag('devtools 已 require 厨房分包入口');
102 _startGame('devtools 跳过等待 loadSubpackage');
103 return;
104 }
105 if (!_nativeApi || typeof _nativeApi.loadSubpackage !== 'function') {
106   if (_requireImagesEntry()) _diag('无 loadSubpackage,已 require 图片入口');
107   if (_requireKitchenEntry()) _diag('无 loadSubpackage,已 require 厨房入口');
108   _startGame('无分包 API,直接启动');
109   return;
110 }
111 _diag('加载图片分包...');
112 var timer = setTimeout(function () {
113   _diag('分包 3s 未回调,继续启动');
114   if (_requireImagesEntry()) _diag('超时后 require 图片入口 OK');
115   if (_requireKitchenEntry()) _diag('超时后 require 厨房入口 OK');
116   _startGame('分包超时兜底');
117 }, 3000);
118 function loadKitchen() {
119   try {
120     _nativeApi.loadSubpackage({
121       name: 'kitchen',
122       success: function () {
123         clearTimeout(timer);
124         _startGame('厨房分包 OK');
125       },
126       fail: function (err) {
127         clearTimeout(timer);
128         _diag('厨房分包失败:' + _errText(err));
129         if (_requireKitchenEntry()) _diag('失败后 require 厨房入口 OK');
130         _startGame('厨房分包失败仍启动');
131       },
132     });
133   } catch (e) {
134     clearTimeout(timer);
135     _diag('厨房 loadSubpackage 异常:' + e);
136     if (_requireKitchenEntry()) _diag('异常后 require 厨房入口 OK');
137     _startGame('厨房分包异常仍启动');
138   }
139 }
140 try {
141   _nativeApi.loadSubpackage({
142     name: 'images',
143     success: function () {
144       _diag('图片分包 OK,加载厨房分包');
145       loadKitchen();
146     },
147     fail: function (err) {
148       _diag('图片分包失败:' + _errText(err));
149       if (_requireImagesEntry()) _diag('失败后 require 图片入口 OK');
150       loadKitchen();
151     }
152   });
153 }
```

```
151     },
152   });
153   } catch (e) {
154     clearTimeout(timer);
155     _diag('loadSubpackage 异常:' + e);
156     _startGame('分包异常仍启动');
157   }
158 }
159 _loadImagesThenStart();
160 setTimeout(function () {
161   if (typeof GameGlobal !== 'undefined' && !GameGlobal.__gameRendered) {
162     _diag('12秒超时 - 游戏未渲染');
163     _showDiag();
164   }
165 }, 12000);
166 /**
167  * 小游戏宿主识别与原生 API 绑定（启动期单一真源，逻辑对齐 src/core/platformDetect.ts）
168  *
169  * - 抖音宿主：注入 tt -> 只走 tt（同时存在的 wx 仅是宿主兼容层，能力不全）
170  * - 微信宿主：注入 wx -> 只走 wx
171  *
172  * 本文件在 game.js 最开头 require，须早于 pxi-adapter 与 game-bundle。
173  */
174 function detectMinigamePlatform() {
175   if (typeof tt !== 'undefined') return 'douyin';
176   if (typeof wx !== 'undefined') return 'wechat';
177   return 'unknown';
178 }
179 function getNativePlatformApi(platform) {
180   var p = platform || detectMinigamePlatform();
181   if (p === 'douyin') return typeof tt !== 'undefined' ? tt : null;
182   if (p === 'wechat') return typeof wx !== 'undefined' ? wx : null;
183   return null;
184 }
185 function canUsePrivacyApi(api, name) {
186   if (!api) return false;
187   if (typeof api.canIUse === 'function') {
188     try { return !!api.canIUse(name); } catch (_) { /* 部分基础库 canIUse 会抛 */ }
189   }
190   return typeof api[name] === 'function';
191 }
192 /** 原生 API 未注册时（devtools / 后台未配隐私政策）的 JS 兜底，避免兼容层 stub 直接抛 unregistered */
193 function makePrivacyFallback(name) {
194   return function (opts) {
195     opts = opts || {};
196     var res = {
197       needAuthorization: false,
198       privacyContractName: "",
199       errMsg: name + ':ok (fallback)',
200     };

```

```
201   if (typeof opts.success === 'function') opts.success(res);
202   if (typeof opts.complete === 'function') opts.complete(res);
203 };
204 }
205 /**
206  * 抖音隐私 API 启动兜底:
207  * 1. 抖音开放平台配置「小游戏隐私政策」后, 原生 API 才会注册 (否则 INTERNAL_APPLY_NATIVE_ERROR)
208  * 2. game.json usePrivacyCheck: true 启用合规链路
209  * 4. 把 wx 兼容层的隐私 stub 代理到 tt; tt 也不可用时降级为 noop (只消噪音, 不改变原生行为)
210  */
211 function initDouyinPrivacyBootstrap() {
212   if (detectMinigamePlatform() !== 'douyin' || typeof tt === 'undefined') return;
213   var privacyNames = [
214     'getPrivacySetting',
215     'requirePrivacyAuthorize',
216     'openPrivacyContract',
217     'onNeedPrivacyAuthorization',
218   ];
219   if (!canUsePrivacyApi(tt, 'getPrivacySetting')) {
220     console.warn(
221       '[Privacy] tt.getPrivacySetting 未注册。处理顺序: '
222       + '①抖音开放平台->设置->基础设置->小游戏隐私政策 填写并发布; '
223       + '②开发者工具升级到 4.2.3+ 并用真机扫码验证 (工具内常报 unregistered) 。',
224     );
225   }
226   if (typeof wx === 'undefined') return;
227   privacyNames.forEach(function (name) {
228     if (name === 'onNeedPrivacyAuthorization') {
229       if (typeof tt[name] === 'function') {
230         wx[name] = function (cb) { return tt[name](cb); };
231       }
232       return;
233     }
234     if (canUsePrivacyApi(tt, name)) {
235       wx[name] = function (opts) { return tt[name](opts); };
236       return;
237     }
238     wx[name] = makePrivacyFallback(name);
239   });
240 }
241 initDouyinPrivacyBootstrap();
242 module.exports = {
243   detectMinigamePlatform: detectMinigamePlatform,
244   getNativePlatformApi: getNativePlatformApi,
245 };
246 import '@core/pixiUnsafeEvalPatch';
247 import { SAVE_KEY } from '@config/CloudConfig';
248 import { EV } from '@config/events';
249 import { EventBus } from '@core/EventBus';
250 import { Game } from '@core/Game';
```

```
251 import { type CloudImportInfo, PersistService } from '@core/PersistService';
252 import { Platform } from '@core/PlatformService';
253 import { SceneManager } from '@core/SceneManager';
254 import { CloudSyncManager } from '@managers/CloudSyncManager';
255 import { SaveManager } from '@managers/SaveManager';
256 import { DestinationScene } from '@scenes/DestinationScene';
257 import { KitchenScene } from '@scenes/KitchenScene';
258 import { LoadingScene } from '@scenes/LoadingScene';
259 import { MarketScene } from '@scenes/MarketScene';
260 import { SpecialMarketScene } from '@scenes/SpecialMarketScene';
261 import { AudioManager } from '@core/AudioManager';
262 import { CdnAssetService } from '@core/CdnAssetService';
263 import { preloadTextures } from '@utils/assets';
264 import { kitchenBootPaths } from '@utils/bootAssets';
265 const BOOT_HOLD_MS = 480;
266 const BOOT_MAX_MS = 12000;
267 function handleCloudSaveReload(info: CloudImportInfo): void {
268   console.warn(
269     `[jiancai] 云端核心存档已覆盖本地，准备刷新 reason=${info.reason}, updatedAt=${info.updatedAt}`,
270   );
271   Platform.showToast('已恢复云端存档，正在刷新', 'none');
272   if (Platform.restartMiniProgram()) return;
273   SaveManager.load();
274   EventBus.emit(EV.kitchenChanged);
275 }
276 async function main(): Promise<void> {
277   const canvas = (typeof GameGlobal !== 'undefined' && GameGlobal.canvas)
278     ? GameGlobal.canvas
279     : (globalThis as any).canvas;
280   if (!canvas) {
281     console.error('[jiancai] 找不到主 canvas');
282     return;
283   }
284   Game.init(canvas);
285   AudioManager.init();
286   const loading = new LoadingScene();
287   SceneManager.register(loading);
288   SceneManager.switchTo('loading');
289   let initialSaveLoaded = false;
290   PersistService.subscribeCloudImport((info) => {
291     if (info.changedKeys.includes(SAVE_KEY)) return;
292     if (initialSaveLoaded) return;
293     handleCloudSaveReload(info);
294   });
295   CloudSyncManager.prewarm();
296   const startupSync = await CloudSyncManager.awaitStartupSync();
297   console.log(`[jiancai] 云同步启动结果: ${startupSync.status}, reason=${startupSync.reason}`);
298   SaveManager.load();
299   initialSaveLoaded = true;
300   const started = Date.now();
```

```
301 const enterKitchen = (): void => {
302   if (SceneManager.current !== loading) return;
303   loading.setProgress(1);
304   SceneManager.register(new KitchenScene());
305   SceneManager.register(new DestinationScene());
306   SceneManager.register(new MarketScene());
307   SceneManager.register(new SpecialMarketScene());
308   SceneManager.switchTo('kitchen');
309 };
310 await CdnAssetService.fetchManifest();
311 const paths = kitchenBootPaths(SaveManager.data);
312 const loaded = preloadTextures(paths, (done, total) => {
313   loading.setProgress(done / Math.max(1, total) * 0.85);
314 });
315 const audioReady = AudioManager.preloadSfx().then() => {
316   loading.setProgress(1);
317 };
318 const timeout = new Promise<void>((resolve) => {
319   globalThis.setTimeout(resolve, BOOT_MAX_MS);
320 });
321 void Promise.race([Promise.all([loaded, audioReady]), timeout]).then() => {
322   const wait = Math.max(0, BOOT_HOLD_MS - (Date.now() - started));
323   globalThis.setTimeout(enterKitchen, wait);
324 });
325 Platform.onHide() => {
326   SaveManager.flush();
327   void CloudSyncManager.flushNow('hide');
328 };
329 console.log(`[jiancai] 启动完成 v${typeof __APP_VERSION__ !== 'undefined' ? __APP_VERSION__ : 'dev'...
330 }
331 void main();
332 /**
333  * CDN 资源配置（微信云存储版，对齐花花）
334  *
335  * 同一 CloudBase 环境多游戏用 filePrefix 隔离。
336  * 游戏代码继续写 minigame 根下的逻辑路径，如 `subpkg_images/market_route_1.jpg`。
337  * 大图走 CDN；首屏厨房 / HUD / 小 UI 留在包内。
338  */
339 export interface CdnConfig {
340   enabled: boolean;
341   appId: string;
342   cloudEnv: string;
343   cloudBucket: string;
344   baseUrl: string;
345   filePrefix: string;
346   cacheRootName: string;
347   downloadRetry: number;
348   downloadTimeoutMs: number;
349   cdnDirs: readonly string[];
350   /** 仅这些前缀算 CDN；同目录里未命中的仍打进微信包。 */
```

```
351   cdnPrefixes: readonly string[];
352   bundledDirs: readonly string[];
353   ignoreFiles: readonly string[];
354 }
355 export const CDN_CONFIG: CdnConfig = {
356   enabled: true,
357   appId: 'wx41ac821080ff87f9',
358   cloudEnv: 'rosa-env-d7grf78r5dbd37323',
359   cloudBucket: '726f-rosa-env-d7grf78r5dbd37323-1414200063',
360   baseUrl: 'https://726f-rosa-env-d7grf78r5dbd37323-1414200063.tcb.qcloud.la',
361   filePrefix: 'jiancai/assets_cdn',
362   cacheRootName: 'jiancai_cdn_cache_v1',
363   downloadRetry: 2,
364   downloadTimeoutMs: 30000,
365   cdnDirs: [
366     'subpkg_images',
367     'subpkg_audio',
368   ],
369   cdnPrefixes: [
370     'subpkg_images/market_route_',
371     'subpkg_images/market_card',
372     'subpkg_images/market_overview',
373     'subpkg_images/stall_rumimage_',
374     'subpkg_images/stall_pile_',
375     'subpkg_images/outing_curtain',
376     'subpkg_images/dest_',
377     'subpkg_images/special_',
378     'subpkg_images/dish_',
379     'subpkg_images npc_',
380     'subpkg_audio/',
381   ],
382   bundledDirs: [
383     'boot',
384     'subpkg_kitchen',
385   ],
386   ignoreFiles: ['game.js', '.DS_Store', 'Thumbs.db', '.gitkeep'],
387 };
388 /**
389  * 统一 HTTP 后端配置 (CloudBase HTTP 访问服务)
390  *
391  * 后端仓库: cloudfunctions/jiancai-api
392  * 环境 ID: rosa-env-d7grf78r5dbd37323 (腾讯云 CloudBase)
393  *
394  * 本游戏代号 (多游戏复用的唯一改动点之一)
395  * 其它游戏复用本套模板时, 改这里 + 后端函数目录名 + CloudBase GAME_KEY 环境变量即可。
396  */
397 export const GAME_KEY = 'jiancai';
398 /** CloudBase HTTP 访问服务根域名 (不含路径) */
399 export const BACKEND_BASE_URL = 'https://rosa-env-d7grf78r5dbd37323.service.tcloudbase.com';
400 /** HTTP 访问服务挂载路径前缀 (与 CloudBase 云函数名一致: cloudfunctions/${GAME_KEY}-api) */
```



```
401 export const BACKEND_PATH_PREFIX = `${GAME_KEY}-api`;
402 export const BACKEND_LOGIN_PATH = `${BACKEND_PATH_PREFIX}/login`;
403 export const BACKEND_PULL_PATH = `${BACKEND_PATH_PREFIX}/save/pull`;
404 export const BACKEND_PUSH_PATH = `${BACKEND_PATH_PREFIX}/save/push`;
405 export const BACKEND_HEALTH_PATH = `${BACKEND_PATH_PREFIX}/health`;
406 /** 请求超时 (毫秒) */
407 export const BACKEND_REQUEST_TIMEOUT_MS = 10000;
408 /** 本地 Token 缓存 key (仅本地, 不纳入云同步) */
409 export const BACKEND_TOKEN_KEY = `${GAME_KEY}_token`;
410 /** 匿名 (H5 / 无平台 code) 场景的稳定设备 ID key */
411 export const BACKEND_ANON_ID_KEY = `${GAME_KEY}_anon_id`;
412 export const SAVE_KEY = `${GAME_KEY}_save`;
413 /** 音量开关只留本地, 不进云同步。 */
414 export const AUDIO_SETTINGS_KEY = `${GAME_KEY}_audio`;
415 export const CLOUD_SYNC_SCHEMA_VERSION = 1;
416 export const CLOUD_SYNC_META_KEY = `${GAME_KEY}_cloud_meta`;
417 /** 云同步白名单: 只有列表里的 key 会被打包上云, 其余仅本地 */
418 export const CLOUD_SYNC_ALLOWLIST = [
419   SAVE_KEY,
420 ] as const;
421 export const CLOUD_SYNC_EXCLUDE_KEYS = [
422   BACKEND_TOKEN_KEY,
423   BACKEND_ANON_ID_KEY,
424   AUDIO_SETTINGS_KEY,
425 ] as const;
426 export const CLOUD_SYNC_STARTUP_TIMEOUT_MS = 2500;
427 export const CLOUD_SYNC_DEBOUNCE_MS = 1500;
428 export const CLOUD_SYNC_BASE_DELAY_MS = 1500;
429 export const CLOUD_SYNC_MAX_BACKOFF_MS = 30000;
430 export const CLOUD_SYNC_MAX_FAIL_COUNT = 5;
431 export const CLOUD_SYNC_RETRY_INTERVAL_MS = 60000;
432 export const CLOUD_SYNC_LOG_THRESHOLD = 3;
433 export type CloudSyncKey = typeof CLOUD_SYNC_ALLOWLIST[number];
434 export const EV = {
435   kitchenChanged: 'kitchen:changed',
436   cookLeveled: 'cook:leveled',
437   recipeUnlocked: 'recipe:unlocked',
438   runChanged: 'run:changed',
439   runExtracted: 'run:extracted',
440   basketChanged: 'basket:changed',
441 } as const;
442 /**
443  * GameKey 平台命名空间 -- 多平台数据隔离标准
444  *
445  * - 微信 / H5: jiancai_{suffix}
446  * - 抖音: jiancai_tt_{suffix}
447  *
448  * 隔离范围: 本地存储 key、云端集合、经分 gameKey。
449  * 不隔离: CloudBase HTTP 路由前缀 (/jiancai-api), 两端共用同一套云函数, 由 JWT 里的 plt 分流。
450  */
```

```
451 import {
452   detectMinigamePlatform,
453   type BackendPlatformCode,
454   type PlatformName,
455 } from '@core/platformDetect';
456 import { GAME_KEY } from './CloudConfig';
457 /** 游戏根标识（云函数名 / HTTP 前缀，不含平台段） */
458 export const BASE_GAME_KEY = GAME_KEY;
459 /** 非微信宿主在 GAME_KEY 与 suffix 之间插入的平台段（当前仅抖音 tt） */
460 export type PlatformScopeSegment = 'tt';
461 const PLATFORM_SCOPE: Partial<Record<PlatformName, PlatformScopeSegment>> = {
462   douyin: 'tt',
463 };
464 const BACKEND_SCOPE: Partial<Record<BackendPlatformCode, PlatformScopeSegment>> = {
465   dy: 'tt',
466 };
467 export function getPlatformScope(
468   platform: PlatformName = detectMinigamePlatform(),
469 ): PlatformScopeSegment | null {
470   return PLATFORM_SCOPE[platform] ?? null;
471 }
472 export function getPlatformScopeFromBackend(platform: string): PlatformScopeSegment | null {
473   const code = String(platform || '').toLowerCase() as BackendPlatformCode;
474   return BACKEND_SCOPE[code] ?? null;
475 }
476 /** 存档 / 集合 / 经分 gameKey 使用的命名空间：jiancai 或 jiancai_tt */
477 export function getScopedGameKey(platform: PlatformName = detectMinigamePlatform()): string {
478   const scope = getPlatformScope(platform);
479   return scope ? `${BASE_GAME_KEY}_${scope}` : BASE_GAME_KEY;
480 }
481 export function getScopedGameKeyFromBackend(platform: string): string {
482   const scope = getPlatformScopeFromBackend(platform);
483   return scope ? `${BASE_GAME_KEY}_${scope}` : BASE_GAME_KEY;
484 }
485 /** jiancai_save / jiancai_tt_save */
486 export function scopedStorageKey(
487   suffix: string,
488   platform: PlatformName = detectMinigamePlatform(),
489 ): string {
490   return `${getScopedGameKey(platform)}_${suffix}`;
491 }
492 /**
493  * 把业务里写死的 `jiancai_xxx` 存储 key 映射到当前平台命名空间。
494  *
495  * 由 PlatformService 的存储方法统一调用，业务代码无需感知：微信原样返回，抖音统一插入 `_tt` 段。
496  * 幂等--已带命名空间的 key 不会被二次加前缀。
497  */
498 export function scopeStorageKey(
499   key: string,
500   platform: PlatformName = detectMinigamePlatform(),
```

```
501 ): string {
502   const scope = getPlatformScope(platform);
503   if (!scope || !key) return key;
504   const scopedPrefix = `${BASE_GAME_KEY}_${scope}`;
505   if (key === scopedPrefix || key.startsWith(`${scopedPrefix}_`)) return key;
506   if (key === BASE_GAME_KEY) return scopedPrefix;
507   if (key.startsWith(`${BASE_GAME_KEY}_`)) {
508     return `${scopedPrefix}_${key.slice(BASE_GAME_KEY.length + 1)}`;
509   }
510   return `${scopedPrefix}_${key}`;
511 }
512 /**
513  * 对齐花花: 逻辑路径一律 `subpkg_audio/*.mp3`, 整目录 CDN。
514  * 播放前 resolveAudioSrc: 真机用 wxfile 缓存, 开发者工具 http://usr 改走 https。
515  * 设 src 后等 onCanplay 再 play; 本地读失败再回落 CDN https。
516  */
517 import { AUDIO_SETTINGS_KEY } from '@config/CloudConfig';
518 import { CdnAssetService } from '@core/CdnAssetService';
519 import { PersistService } from '@core/PersistService';
520 import { Platform } from '@core/PlatformService';
521 export const SFX_IDS = [
522   'ui_click',
523   'ui_open',
524   'ui_close',
525   'ui_deny',
526   'coin_gain',
527   'coin_spend',
528   'fridge_open',
529   'cook_sizzle',
530   'cook_done',
531   'eat',
532   'level_up',
533   'upgrade',
534   'recipe_paper',
535   'outing',
536   'walk',
537   'rummage',
538   'item_reveal',
539   'pickup_veg',
540   'pickup_wet',
541   'pickup_god',
542   'basket_place',
543   'basket_discard',
544   'gather',
545   'event_pop',
546   'result_safe',
547   'result_dusk',
548 ] as const;
549 export type SfxId = typeof SFX_IDS[number];
550 export type MarketBgmId =
```

```
551 | 'market_xiangko'
552 | 'market_heyang'
553 | 'market_shanwu'
554 | 'market_jiangbian'
555 | 'market_laocheng';
556 export type BgmId = 'kitchen' | 'outing' | MarketBgmId;
557 const MARKET_BGM: Record<string, MarketBgmId> = {
558   xiangko: 'market_xiangko',
559   heyang: 'market_heyang',
560   qiaotou: 'market_heyang',
561   shanwu: 'market_shanwu',
562   jiangbian: 'market_jiangbian',
563   nanshi: 'market_laocheng',
564   laocheng: 'market_laocheng',
565   dukou: 'market_jiangbian',
566   shanzhen: 'market_laocheng',
567 };
568 const AUDIO_DIR = 'subpkg_audio';
569 const BGM_VOL = 0.42;
570 const SFX_VOL = 0.86;
571 /** 捡到菜 / 结算: 压过 BGM 的爽感短句 */
572 const REWARD_SFX = new Set<SfxId>(['item_reveal', 'result_safe', 'result_dusk', 'pickup_god', 'le...
573 const REWARD_VOL = 1;
574 const TAG = '[Audio]';
575 interface AudioSettings {
576   musicEnabled: boolean;
577   soundEnabled: boolean;
578 }
579 type WxInnerAudioContext = {
580   src: string;
581   loop: boolean;
582   volume: number;
583   autoplay?: boolean;
584   obeyMuteSwitch?: boolean;
585   play: () => void;
586   pause: () => void;
587   stop?: () => void;
588   destroy?: () => void;
589   seek?: (position: number) => void;
590   onError?: (handler: (error: unknown) => void) => void;
591   onEnded?: (handler: () => void) => void;
592   onCanplay?: (handler: () => void) => void;
593   onPlay?: (handler: () => void) => void;
594 };
595 interface AudioSettingsState {
596   musicEnabled: boolean;
597   soundEnabled: boolean;
598 }
599 class AudioManagerClass {
600   private wxBgm: WxInnerAudioContext | null = null;
```

```
601 private webBgm: HTMLAudioElement | null = null;
602 private initialized = false;
603 private firstGestureBound = false;
604 private hidden = false;
605 private musicEnabled = this.readSettings().musicEnabled;
606 private soundEnabled = this.readSettings().soundEnabled;
607 private bgmId: BgmId | null = null;
608 private bgmSrc = "";
609 private bgmLogical = "";
610 private bgmRequestSeq = 0;
611 private bgmHttpsTried = false;
612 private readonly webSfxPool = new Map<string, HTMLAudioElement>();
613 sfxPath(id: SfxId): string {
614     return `${AUDIO_DIR}/${id}.mp3`;
615 }
616 bgmPath(id: BgmId): string {
617     return `${AUDIO_DIR}/bgm_${id}.mp3`;
618 }
619 init(): void {
620     if (this.initialized) return;
621     this.initialized = true;
622     this.bindFirstGestureReplay();
623     Platform.onHide() => {
624         this.hidden = true;
625         this.pauseBackgroundMusic();
626     });
627     Platform.onShow() => {
628         this.hidden = false;
629         if (this.musicEnabled) this.playBackgroundMusic();
630     });
631 }
632 preloadSfx(): Promise<void> {
633     this.init();
634     return CdnAssetService.preloadPaths(SFX_IDS.map((id) => this.sfxPath(id))).catch((err) => {
635         console.warn(TAG, '音效预加载失败', err);
636     });
637 }
638 isMusicEnabled(): boolean {
639     return this.musicEnabled;
640 }
641 isSoundEnabled(): boolean {
642     return this.soundEnabled;
643 }
644 isMuted(): boolean {
645     return !this.musicEnabled && !this.soundEnabled;
646 }
647 setMuted(muted: boolean): void {
648     this.musicEnabled = !muted;
649     this.soundEnabled = !muted;
650     this.writeSettings();
```

```
651   if (muted) this.pauseBackgroundMusic();
652   else this.playBackgroundMusic();
653 }
654 toggleMuted(): void {
655   this.setMuted(!this.isMuted());
656 }
657 play(id: SfxId, volume = REWARD_SFX.has(id) ? REWARD_VOL : SFX_VOL): void {
658   if (!this.soundEnabled) return;
659   this.init();
660   const logical = this.sfxPath(id);
661   void CdnAssetService.resolveAudioSrc(logical)
662     .then((src) => this.playResolvedSfx(id, src, volume, logical))
663     .catch((err) => {
664       console.warn(TAG, `音效 "${id}" 资源未就绪`, err);
665     });
666 }
667 playPickup(zone: 'dry' | 'wet'): void {
668   this.play(zone === 'wet' ? 'pickup_wet' : 'pickup_veg');
669 }
670 /** 任意途径有物品出现：同一首出现音。神捡演出另走 pickup_god。 */
671 playGain(_opts?: { god?: boolean }): void {
672   this.play('item_reveal');
673 }
674 playMarketBgm(marketId: string): void {
675   this.playBgm(MARKET_BGM[marketId] ?? 'market_xiangko');
676 }
677 playBgm(id: BgmId): void {
678   this.init();
679   const logical = this.bgmPath(id);
680   this.bgmId = id;
681   this.bgmLogical = logical;
682   this.bgmHttpsTried = false;
683   const seq = ++this.bgmRequestSeq;
684   void CdnAssetService.resolveAudioSrc(logical)
685     .then((src) => {
686       if (seq !== this.bgmRequestSeq) return;
687       this.switchBackgroundMusic(src);
688     })
689     .catch((err) => {
690       if (seq !== this.bgmRequestSeq) return;
691       console.warn(TAG, `BGM "${logical}" 资源未就绪`, err);
692     });
693 }
694 playBackgroundMusic(): void {
695   if (!this.musicEnabled || this.hidden || !this.bgmSrc) return;
696   this.init();
697   try {
698     this.wxBgm?.play();
699     void this.webBgm?.play?.();
700   } catch (_) {}
```

```
701 }
702 pauseBackgroundMusic(): void {
703   try {
704     this.wxBgm?.pause();
705     this.webBgm?.pause?.();
706   } catch (_) {}
707 }
708 private playResolvedSfx(name: string, src: string, volume: number, logical?: string): void {
709   const wxCtx = Platform.createInnerAudioContext() as WxInnerAudioContext | null;
710   if (wxCtx) {
711     this.playWxSfx(name, wxCtx, src, volume, logical);
712     return;
713   }
714   if (typeof Audio !== 'undefined') this.playWebSfx(src, volume);
715 }
716 private playWxSfx(
717   name: string,
718   audio: WxInnerAudioContext,
719   src: string,
720   volume: number,
721   logical?: string,
722 ): void {
723   let done = false;
724   let started = false;
725   let usedHttps = false;
726   const https = logical && CdnAssetService.isCdnPath(logical) ? CdnAssetService.publicUrl(logic...
727   const cleanup = () => {
728     if (done) return;
729     done = true;
730     try { audio.destroy?.(); } catch (_) {}
731   };
732   const tryPlay = () => {
733     if (done || started) return;
734     started = true;
735     try {
736       audio.play();
737     } catch (error) {
738       console.warn(TAG, `音效 "${name}" play()`, error);
739       cleanup();
740     }
741   };
742   audio.loop = false;
743   audio.volume = Math.max(0, Math.min(1, volume));
744   audio.obeyMuteSwitch = false;
745   audio.onError?.((error) => {
746     if (!usedHttps && https && src !== https) {
747       usedHttps = true;
748       started = false;
749       try {
750         audio.src = https;
```

```
751     setTimeout(() => tryPlay(), typeof audio.onCanplay === 'function' ? 300 : 0);
752   } catch (_) {
753     cleanup();
754   }
755   return;
756 }
757 console.warn(TAG, `音效 "${name}" 播放失败`, error);
758 cleanup();
759 });
760 audio.onEnded?.(() => cleanup());
761 if (typeof audio.onCanplay === 'function') {
762   audio.onCanplay(() => tryPlay());
763 }
764 audio.src = src;
765 setTimeout(() => tryPlay(), typeof audio.onCanplay === 'function' ? 300 : 0);
766 }
767 private playWebSfx(src: string, volume: number): void {
768   let sfx = this.webSfxPool.get(src);
769   if (!sfx) {
770     sfx = new Audio(src);
771     sfx.preload = 'auto';
772     this.webSfxPool.set(src, sfx);
773   }
774   try {
775     sfx.volume = Math.max(0, Math.min(1, volume));
776     sfx.currentTime = 0;
777     void sfx.play();
778   } catch (_) {}
779 }
780 private bindFirstGestureReplay(): void {
781   if (this.firstGestureBound) return;
782   this.firstGestureBound = true;
783   const api = Platform.api;
784   if (api?.onTouchStart) {
785     api.onTouchStart(() => this.playBackgroundMusic());
786     return;
787   }
788   const root = globalThis as any;
789   root.addEventListener?.('pointerdown', () => this.playBackgroundMusic(), { once: true });
790   root.addEventListener?.('touchstart', () => this.playBackgroundMusic(), { once: true });
791 }
792 private switchBackgroundMusic(src: string): void {
793   this.bgmSrc = src;
794   if (!this.initialized) this.init();
795   try {
796     if (!this.wxBgm) {
797       const ctx = Platform.createInnerAudioContext() as WxInnerAudioContext | null;
798       if (ctx) {
799         ctx.loop = true;
800         ctx.volume = BGM_VOL;
```



```
801     ctx.autoplay = false;
802     ctx.obeyMuteSwitch = false;
803     ctx.onError?.((error) => {
804         const https = this.bgmLogical && CdnAssetService.isCdnPath(this.bgmLogical)
805             ? CdnAssetService.publicUrl(this.bgmLogical)
806             : "";
807         if (!this.bgmHttpsTried && https && this.wxBgm && this.wxBgm.src !== https) {
808             this.bgmHttpsTried = true;
809             try {
810                 this.wxBgm.src = https;
811                 this.bgmSrc = https;
812             } catch (_) {}
813             return;
814         }
815         console.warn(TAG, 'BGM failed', this.bgmSrc || src, error);
816     });
817     if (typeof ctx.onCanplay === 'function') {
818         ctx.onCanplay(() => this.playBackgroundMusic());
819     }
820     this.wxBgm = ctx;
821 }
822 }
823 if (this.wxBgm) {
824     this.wxBgm.pause();
825     this.wxBgm.src = src;
826     this.wxBgm.loop = true;
827     this.wxBgm.volume = BGM_VOL;
828 }
829 if (typeof Audio !== 'undefined' && !this.wxBgm) {
830     if (!this.webBgm) this.webBgm = new Audio(src);
831     else this.webBgm.src = src;
832     this.webBgm.loop = true;
833     this.webBgm.volume = BGM_VOL;
834     this.webBgm.load();
835 }
836 } catch (_) {}
837 setTimeout(() => this.playBackgroundMusic(), this.wxBgm && typeof this.wxBgm.onCanplay === 'f...
838 }
839 private readSettings(): AudioSettingsState {
840     const stored = PersistService.readJSON<Partial<AudioSettings>>>(AUDIO_SETTINGS_KEY);
841     return {
842         musicEnabled: stored?.musicEnabled !== false,
843         soundEnabled: stored?.soundEnabled !== false,
844     };
845 }
846 private writeSettings(): void {
847     PersistService.writeJSON(AUDIO_SETTINGS_KEY, {
848         musicEnabled: this.musicEnabled,
849         soundEnabled: this.soundEnabled,
850     }, { markDirty: false });
```

```
851 }
852 }
853 const holder = typeof GameGlobal !== 'undefined' ? GameGlobal : globalThis;
854 if (!(holder as any).__jiancaiAudioManager) {
855   (holder as any).__jiancaiAudioManager = new AudioManagerClass();
856 }
857 export const AudioManager: AudioManagerClass = (holder as any).__jiancaiAudioManager;
858 /**
859  * 统一后端访问层（CloudBase HTTP 访问服务）
860  *
861  * - 跨平台：复用 PlatformService.request()（微信 wx.request / 抖音 tt.request / H5 fetch）
862  * - 身份：微信/抖音走原生 code 登录，H5 等走本地匿名 ID 登录；userId 形如 `wx:{openid}` / `dy:{openid}` / `anon:{uuid}`
863  * - Token：服务端签发 JWT，客户端缓存在本地；过期或 401 自动重新 login
864  */
865 import {
866   BACKEND_ANON_ID_KEY,
867   BACKEND_BASE_URL,
868   BACKEND_LOGIN_PATH,
869   BACKEND_PULL_PATH,
870   BACKEND_PUSH_PATH,
871   BACKEND_REQUEST_TIMEOUT_MS,
872   BACKEND_TOKEN_KEY,
873 } from '@config/CloudConfig';
874 import { Platform } from './PlatformService';
875 export interface BackendPullResult {
876   userId: string;
877   platform: string;
878   exists: boolean;
879   schemaVersion: number;
880   updatedAt: number;
881   payload: Record<string, string>;
882   payloadKeys: string[];
883   clientFingerprint?: string;
884 }
885 export interface BackendPushPayload {
886   schemaVersion: number;
887   updatedAt: number;
888   /** 本地变更所基于的云端版本，用于服务端拒绝旧缓存覆盖。 */
889   baseRemoteUpdatedAt: number;
890   clientFingerprint: string;
891   payload: Record<string, string>;
892 }
893 export interface BackendPushResult {
894   userId: string;
895   updatedAt: number;
896   savedAt: number;
897   mode: 'insert' | 'update';
898   sizeBytes: number;
899 }
900 interface StoredToken {
```

```
901 token: string;
902 userId: string;
903 platform: string;
904 expiresAt: number;
905 }
906 /** 业务错误（非网络异常），携带 HTTP 状态码与业务错误码 */
907 export class BackendError extends Error {
908     readonly status: number;
909     readonly code: string;
910     readonly data?: any;
911     constructor(status: number, code: string, message: string, data?: any) {
912         super(message);
913         this.name = 'BackendError';
914         this.status = status;
915         this.code = code;
916         this.data = data;
917     }
918 }
919 class BackendServiceClass {
920     private _stored: StoredToken | null = null;
921     private _loginInflight: Promise<StoredToken> | null = null;
922     get available(): boolean {
923         return Platform.canUseBackend;
924     }
925     get userId(): string {
926         return this._stored?.userId || '';
927     }
928     get platform(): string {
929         return this._stored?.platform || '';
930     }
931     /** 拿到有效 token；过期 / 无缓存会自动登录 */
932     async ensureToken(): Promise<StoredToken> {
933         if (this._stored && this._stored.expiresAt - Date.now() > 60_000) {
934             return this._stored;
935         }
936         const cached = this._loadTokenFromStorage();
937         if (cached && cached.expiresAt - Date.now() > 60_000) {
938             this._stored = cached;
939             return cached;
940         }
941         if (this._loginInflight) return this._loginInflight;
942         this._loginInflight = this._login()
943             .then((t) => {
944                 this._loginInflight = null;
945                 return t;
946             })
947             .catch((e) => {
948                 this._loginInflight = null;
949                 throw e;
950             });
```

```
951   return this._loginInflight;
952 }
953 async pullSave(): Promise<BackendPullResult> {
954   return this._callWithAuth<BackendPullResult>(BACKEND_PULL_PATH, {});
955 }
956 async pushSave(snapshot: BackendPushPayload): Promise<BackendPushResult> {
957   return this._callWithAuth<BackendPushResult>(BACKEND_PUSH_PATH, snapshot);
958 }
959 /** 清空本地 token（排障用），不影响本地存档 */
960 clearToken(): void {
961   this._stored = null;
962   try {
963     Platform.removeStorageSync(BACKEND_TOKEN_KEY);
964   } catch (_) {}
965 }
966 private async _login(): Promise<StoredToken> {
967   const body = await this._buildLoginBody();
968   const { status, data } = await this._request(BACKEND_LOGIN_PATH, body, undefined);
969   if (status !== 200 || !data || data.ok !== true || !data.data?.token) {
970     const code = data?.code || 'LOGIN_FAIL';
971     const msg = data?.error || `login failed (status=${status})`;
972     throw new BackendError(status, code, msg, data?.data);
973   }
974   const stored: StoredToken = {
975     token: String(data.data.token),
976     userId: String(data.data.userId || ''),
977     platform: String(data.data.platform || body.platform),
978     expiresAt: Number(data.data.expiresAt || 0),
979   };
980   this._stored = stored;
981   this._saveTokenToStorage(stored);
982   console.log('[Backend] login ok userId=${stored.userId}');
983   return stored;
984 }
985 private async _buildLoginBody(): Promise<{ platform: string; code?: string; anonId?: string }> {
986   if (Platform.isWechat) {
987     const code = await Platform.loginCode();
988     if (!code) throw new BackendError(0, 'NO_WX_CODE', 'wx.login 未返回 code');
989     return { platform: 'wx', code };
990   }
991   if (Platform.isDouyin) {
992     const code = await Platform.loginCode();
993     if (!code) throw new BackendError(0, 'NO_TT_CODE', 'tt.login 未返回 code');
994     return { platform: 'dy', code };
995   }
996   const anonId = this._getOrCreateAnonId();
997   return { platform: 'anon', anonId };
998 }
999 private _getOrCreateAnonId(): string {
1000   let id = '';
```

```
1001   try {
1002     id = Platform.getStorageSync(BACKEND_ANON_ID_KEY) || '';
1003   } catch (_) {}
1004   if (id) return id;
1005   id = 'anon_' + Date.now().toString(36) + '_' + Math.random().toString(36).slice(2, 10);
1006   try {
1007     Platform.setStorageSync(BACKEND_ANON_ID_KEY, id);
1008   } catch (_) {}
1009   return id;
1010 }
1011 private async _callWithAuth<T>(path: string, body: any): Promise<T> {
1012   const token = await this.ensureToken();
1013   const { status, data } = await this._request(path, body, token.token);
1014   if (status === 401) {
1015     this.clearToken();
1016     const retryToken = await this.ensureToken();
1017     const res2 = await this._request(path, body, retryToken.token);
1018     return this._unwrap<T>(res2.status, res2.data);
1019   }
1020   return this._unwrap<T>(status, data);
1021 }
1022 private _unwrap<T>(status: number, data: any): T {
1023   if (status === 200 && data && data.ok === true) {
1024     return data.data as T;
1025   }
1026   const code = data?.code || `HTTP_${status}`;
1027   const msg = data?.error || `request failed status=${status}`;
1028   throw new BackendError(status, code, msg, data?.data);
1029 }
1030 private async _request(
1031   path: string,
1032   body: any,
1033   token: string | undefined,
1034 ): Promise<{ status: number; data: any }> {
1035   const url = BACKEND_BASE_URL + path;
1036   const headers: Record<string, string> = {
1037     'content-type': 'application/json',
1038   };
1039   if (token) headers['authorization'] = `Bearer ${token}`;
1040   const res = await Platform.request({
1041     url,
1042     method: 'POST',
1043     data: body || {},
1044     headers,
1045     timeoutMs: BACKEND_REQUEST_TIMEOUT_MS,
1046   });
1047   return { status: res.statusCode, data: res.data };
1048 }
1049 private _loadTokenFromStorage(): StoredToken | null {
1050   try {
```

```

1051     const raw = Platform.getStorageSync(BACKEND_TOKEN_KEY);
1052     if (!raw) return null;
1053     const parsed = JSON.parse(raw);
1054     if (!parsed || typeof parsed.token !== 'string' || !parsed.token) return null;
1055     if (parsed.platform && parsed.platform !== Platform.backendPlatformCode) {
1056         console.warn(
1057             `[Backend] 丢弃平台不匹配的 token: stored=${JSON.stringify(parsed.platform)}, current=${JSON.stringify(Platform.backendPlatformCode)}`
1058         );
1059         return null;
1060     }
1061     return {
1062         token: parsed.token,
1063         userId: parsed.userId || '',
1064         platform: parsed.platform || '',
1065         expiresAt: Number(parsed.expiresAt || 0),
1066     };
1067 } catch (_) {
1068     return null;
1069 }
1070 }
1071 private _saveTokenToStorage(stored: StoredToken): void {
1072     try {
1073         Platform.setStorageSync(BACKEND_TOKEN_KEY, JSON.stringify(stored));
1074     } catch (_) {}
1075 }
1076 }
1077 export const BackendService = new BackendServiceClass();
1078 import { CDN_CONFIG, type CdnConfig } from '@config/CdnConfig';
1079 import { Platform } from './PlatformService';
1080 export interface CdnManifestFile {
1081     hash?: string;
1082     size?: number;
1083 }
1084 export interface CdnManifest {
1085     version?: number;
1086     updated?: string;
1087     files: Record<string, CdnManifestFile>;
1088 }
1089 type ProgressCallback = (loaded: number, total: number) => void;
1090 class CdnAssetServiceClass {
1091     private readonly _config: CdnConfig = CDN_CONFIG;
1092     private readonly _cdnDirPrefixes = this._config.cdnDirs.map(d => this._prefix(d));
1093     private readonly _bundledPrefixes = this._config.bundledDirs.map(d => this._prefix(d));
1094     private _manifest: CdnManifest | null = null;
1095     private _manifestReady = false;
1096     private _downloadQueue = new Map<string, Promise<boolean>>();
1097     private _localExistsCache = new Map<string, boolean>();
1098     private _accessLog = new Map<string, number>();
1099     private _accessFrame = 0;
1100     get enabled(): boolean {

```

```
1101     return this._config.enabled;
1102   }
1103   get manifestReady(): boolean {
1104     return this._manifestReady;
1105   }
1106   get manifest(): CdnManifest | null {
1107     return this._manifest;
1108   }
1109   isCdnPath(path: string): boolean {
1110     const normalized = this._normalize(path);
1111     if (!this.enabled || this.isBundledPath(normalized)) return false;
1112     return this._matchesCdnPrefix(normalized);
1113   }
1114   isBundledPath(path: string): boolean {
1115     const normalized = this._normalize(path);
1116     return this._bundledPrefixes.some(prefix => normalized.startsWith(prefix));
1117   }
1118   areAllCdnPaths(paths: readonly string[]): boolean {
1119     return paths.length > 0 && paths.every(path => this.isCdnPath(path));
1120   }
1121   resolveAsset(path: string): string | null {
1122     const logicalPath = this._normalize(path);
1123     if (!this.isCdnPath(logicalPath)) return logicalPath;
1124     this._touch(logicalPath);
1125     if (this._isCacheValid(logicalPath)) return this._getCachePath(logicalPath);
1126     return null;
1127   }
1128   async resolveOrDownload(path: string): Promise<string> {
1129     const logicalPath = this._normalize(path);
1130     const resolved = this.resolveAsset(logicalPath);
1131     if (resolved) return resolved;
1132     if (!this.isCdnPath(logicalPath)) {
1133       return logicalPath;
1134     }
1135     const ok = await this.download(logicalPath);
1136     if (ok && this._isCacheValid(logicalPath)) return this._getCachePath(logicalPath);
1137     // CDN 专属资源不在微信包里。再回落相对路径只会让图片/音效二次失败。
1138     if (this._config.baseUrl) {
1139       console.warn(`[CDN] 本地缓存未就绪，直连云端: ${logicalPath}`);
1140       return this._getCdnUrl(logicalPath);
1141     }
1142     console.warn(`[CDN] 资源未从云端就绪，使用本地分包路径: ${logicalPath}`);
1143     return logicalPath;
1144   }
1145   /** 开发者工具 USER_DATA_PATH 是 http://usr，图片能读，InnerAudio 必须 wxfile://。 */
1146   toInnerAudioSrc(path: string): string {
1147     if (/^https?:\\\/usr\\\/.test(path)) return path.replace(/^https?:\\\/usr\\\/, 'wxfile://usr/');
1148     return path;
1149   }
1150   publicUrl(path: string): string {
```

```
1151   return this._getCdnUrl(this._normalize(path));
1152 }
1153 /** 给 InnerAudio: 优先本地 wxfile; 开发者工具 http://usr 必须改走 https, 否则 set src request:fail. */
1154 async resolveAudioSrc(path: string): Promise<string> {
1155   const logicalPath = this._normalize(path);
1156   const resolved = await this.resolveOrDownload(logicalPath);
1157   if (/^https?:\//usr\/.test(resolved) && this._config.baseUrl && this.isCdnPath(logicalPath)) {
1158     return this.publicUrl(logicalPath);
1159   }
1160   const playable = this.toInnerAudioSrc(resolved);
1161   if (
1162     playable.startsWith('wxfile://')
1163     || playable.startsWith('https://')
1164     || playable.startsWith('http://tmp')
1165   ) {
1166     return playable;
1167   }
1168   if (this.isCdnPath(logicalPath) && this._config.baseUrl) return this.publicUrl(logicalPath);
1169   return playable;
1170 }
1171 invalidateCache(path: string): void {
1172   const logicalPath = this._normalize(path);
1173   const cachePath = this._getCachePath(logicalPath);
1174   this._localExistsCache.delete(cachePath);
1175   const fs = this._getFs();
1176   try { fs?.unlinkSync(cachePath); } catch (_) {}
1177   try { fs?.unlinkSync(`${cachePath}.meta`); } catch (_) {}
1178 }
1179 async fetchManifest(): Promise<boolean> {
1180   if (!this.enabled) {
1181     this._manifest = { files: {} };
1182     this._manifestReady = true;
1183     return false;
1184   }
1185   const fs = this._getFs();
1186   if (!fs || !this._config.baseUrl) {
1187     this._loadCachedManifest();
1188     return false;
1189   }
1190   /** 不要走 wx.downloadFile 拉 manifest.json。
1191    * 开发者工具/基础库对 CloudBase 默认域名 (*.tcb.qcloud.la) 上的 application/json
1192    * 会触发云开发协议层 JSON.parse, 普通静态 JSON 会被报成
1193    * "downloadFile:fail Unexpected end of JSON input"。
1194    * 改走 wx.request 直接拿文本。 */
1195   const url = `${this._getCdnUrl('manifest.json')}?_t=${Date.now()}`;
1196   try {
1197     const text = await this._requestText(url);
1198     if (!text) {
1199       this._loadCachedManifest();
1200       return false;
```



```
1201     }
1202     this._manifest = JSON.parse(text) as CdnManifest;
1203     this._manifestReady = true;
1204     this._ensureCacheDir(this._getCachePath('manifest.json'));
1205     try { fs.writeFileSync(this._getCachePath('manifest.json'), text, 'utf-8'); } catch (_) {}
1206     return true;
1207 } catch (e) {
1208     console.warn('[CDN] manifest 拉取失败, 使用本地缓存:', e);
1209     this._loadCachedManifest();
1210     return false;
1211 }
1212 }
1213 async download(path: string): Promise<boolean> {
1214     const logicalPath = this._normalize(path);
1215     if (!this.isCdnPath(logicalPath)) return true;
1216     if (this._isCacheValid(logicalPath)) return true;
1217     const inflight = this._downloadQueue.get(logicalPath);
1218     if (inflight) return inflight;
1219     const task = this._downloadWithRetry(logicalPath).finally(() => {
1220         this._downloadQueue.delete(logicalPath);
1221     });
1222     this._downloadQueue.set(logicalPath, task);
1223     return task;
1224 }
1225 async preloadPaths(paths: readonly string[], onProgress?: ProgressCallback): Promise<void> {
1226     const cdnPaths = paths
1227         .map(p => this._normalize(p))
1228         .filter(p => this.isCdnPath(p) && !this._isCacheValid(p));
1229     if (cdnPaths.length === 0) {
1230         onProgress?.(paths.length, paths.length);
1231         return;
1232     }
1233     let done = 0;
1234     await Promise.race([
1235         Promise.all(cdnPaths.map(async (p) => {
1236             await this.download(p);
1237             done++;
1238             onProgress?.(paths.length - cdnPaths.length + done, paths.length);
1239         })),
1240         new Promise<void>(resolve => setTimeout(resolve, this._config.downloadTimeoutMs)),
1241     ]);
1242 }
1243 clearCache(): void {
1244     const fs = this._getFs();
1245     if (!fs || this._accessLog.size === 0) return;
1246     const entries = [...this._accessLog.entries()].sort((a, b) => a[1] - b[1]);
1247     const evictCount = Math.ceil(entries.length * 0.2);
1248     for (let i = 0; i < evictCount; i++) {
1249         const logicalPath = entries[i][0];
1250         const cachePath = this._getCachePath(logicalPath);
```

```
1251     try { fs.unlinkSync(cachePath); } catch (_) {}
1252     try { fs.unlinkSync(`${cachePath}.meta`); } catch (_) {}
1253     this._localExistsCache.delete(cachePath);
1254     this._accessLog.delete(logicalPath);
1255   }
1256 }
1257 clearAllCache(): void {
1258   const fs = this._getFs();
1259   const root = this._getCacheRootPath();
1260   if (!fs || !root) return;
1261   this._localExistsCache.clear();
1262   this._accessLog.clear();
1263   try {
1264     fs.rmdirSync(root, true);
1265     console.log('[CDN] 已清空本地 CDN 缓存');
1266   } catch (e) {
1267     console.warn('[CDN] 清空本地 CDN 缓存失败:', e);
1268   }
1269 }
1270 private async _downloadWithRetry(logicalPath: string): Promise<boolean> {
1271   const fs = this._getFs();
1272   if (!fs || !this._config.baseUrl) return false;
1273   const cachePath = this._getCachePath(logicalPath);
1274   this._ensureCacheDir(cachePath);
1275   for (let attempt = 0; attempt <= this._config.downloadRetry; attempt++) {
1276     try {
1277       const res = await this._downloadUrl(this._getCdnUrl(logicalPath));
1278       if (!res.tempFilePath) throw new Error('downloadFile missing tempFilePath');
1279       fs.copyFileSync(res.tempFilePath, cachePath);
1280       const size = this._getLocalFileSize(cachePath);
1281       const expected = this._manifest?.files?.[logicalPath]?.size;
1282       if (size < 64 || (typeof expected === 'number' && expected > 0 && size !== expected)) {
1283         this._localExistsCache.set(cachePath, false);
1284         try { fs.unlinkSync(cachePath); } catch (_) {}
1285         try { fs.unlinkSync(`${cachePath}.meta`); } catch (_) {}
1286         throw new Error(`bad size ${size} expected ${expected} || '>=64'`);
1287       }
1288       this._localExistsCache.set(cachePath, true);
1289       const hash = this._manifest?.files?.[logicalPath]?.hash || '';
1290       try { fs.writeFileSync(`${cachePath}.meta`, hash, 'utf-8'); } catch (_) {}
1291       return true;
1292     } catch (e) {
1293       const msg = e instanceof Error ? e.message : String(e);
1294       if (msg.includes('status=404') || attempt >= this._config.downloadRetry) {
1295         console.warn('[CDN] 下载失败 ${logicalPath}:', e);
1296         return false;
1297       }
1298       await new Promise(resolve => setTimeout(resolve, 500 * (attempt + 1)));
1299     }
1300   }
```

```
1301     return false;
1302 }
1303 private _requestText(url: string): Promise<string> {
1304     return Platform.requestText(url, this._config.downloadTimeoutMs)
1305         .catch((err: unknown) => {
1306             const msg = err instanceof Error ? err.message : String(err);
1307             console.warn('[CDN] request fail: ${url}, ${msg}');
1308             throw err instanceof Error ? err : new Error(msg);
1309         });
1310 }
1311 private _downloadUrl(url: string): Promise<{ tempFilePath?: string }> {
1312     return Platform.downloadFile({ url })
1313         .catch((err: unknown) => {
1314             const msg = err instanceof Error ? err.message : String(err);
1315             console.warn('[CDN] downloadFile fail: ${url}, ${msg}');
1316             throw err instanceof Error ? err : new Error(msg);
1317         });
1318 }
1319 private _loadCachedManifest(): void {
1320     const fs = this._getFs();
1321     try {
1322         const text = fs?.readFileSync(this._getCachePath('manifest.json'), 'utf-8');
1323         this._manifest = text ? JSON.parse(text) as CdnManifest : { files: {} };
1324     } catch (_) {
1325         this._manifest = { files: {} };
1326     }
1327     this._manifestReady = true;
1328 }
1329 private _isCacheValid(logicalPath: string): boolean {
1330     if (!this._cacheFileExists(logicalPath)) return false;
1331     const localSize = this._getLocalFileSize(this._getCachePath(logicalPath));
1332     if (localSize < 64) return false;
1333     const entry = this._manifest?.files?.[logicalPath];
1334     if (entry?.size && localSize !== entry.size) {
1335         return false;
1336     }
1337     if (!entry?.hash) return true;
1338     return this._readCachedHash(logicalPath) === entry.hash;
1339 }
1340 private _cacheFileExists(logicalPath: string): boolean {
1341     return this._localFileExists(this._getCachePath(logicalPath));
1342 }
1343 private _localFileExists(path: string): boolean {
1344     const cached = this._localExistsCache.get(path);
1345     if (cached !== undefined) return cached;
1346     const fs = this._getFs();
1347     if (!fs) {
1348         this._localExistsCache.set(path, false);
1349         return false;
1350     }
```

```
1351     try {
1352         fs.accessSync(path);
1353         this._localExistsCache.set(path, true);
1354         return true;
1355     } catch (_) {
1356         this._localExistsCache.set(path, false);
1357         return false;
1358     }
1359 }
1360 private _getLocalFileSize(path: string): number {
1361     const fs = this._getFs();
1362     try {
1363         const stat = fs?.statSync(path);
1364         return Number(stat?.size || 0);
1365     } catch (_) {
1366         return 0;
1367     }
1368 }
1369 private _readCachedHash(logicalPath: string): string | null {
1370     const fs = this._getFs();
1371     try {
1372         return String(fs?.readFileSync(`${this._getCachePath(logicalPath)}.meta`, 'utf-8') || '').t...
1373     } catch (_) {
1374         return null;
1375     }
1376 }
1377 private _getCdnUrl(logicalPath: string): string {
1378     const base = this._config.baseUrl.replace(/\/+$/, '');
1379     return `${base}/${this._config.filePrefix}/${logicalPath}`;
1380 }
1381 private _getCachePath(logicalPath: string): string {
1382     return `${this._getCacheRootPath()}/${logicalPath}`;
1383 }
1384 private _getCacheRootPath(): string {
1385     const userDataPath = this._getUserDataPath();
1386     return userDataPath ? `${userDataPath}/${this._config.cacheRootName}` : '';
1387 }
1388 private _ensureCacheDir(filePath: string): void {
1389     const fs = this._getFs();
1390     const userDataPath = this._getUserDataPath();
1391     if (!fs || !userDataPath) return;
1392     const dir = filePath.split('/').slice(0, -1).join('/');
1393     try { fs.accessSync(dir); return; } catch (_) {}
1394     const segments = dir.replace(`${userDataPath}/`, '').split('/').filter(Boolean);
1395     let cur = userDataPath;
1396     for (const seg of segments) {
1397         cur += `/${seg}`;
1398         try { fs.accessSync(cur); } catch (_) {
1399             try { fs.mkdirSync(cur, true); } catch (_) {}
1400         }
```

```
1401     }
1402   }
1403   private _getFs(): any {
1404     return Platform.getFileSystemManager();
1405   }
1406   private _getUserDataPath(): string {
1407     return Platform.userDataPath;
1408   }
1409   private _touch(logicalPath: string): void {
1410     this._accessFrame++;
1411     this._accessLog.set(logicalPath, this._accessFrame);
1412   }
1413   private _matchesCdnPrefix(path: string): boolean {
1414     if (this._config.cdnPrefixes.length > 0) {
1415       return this._config.cdnPrefixes.some(prefix => this._matchPrefix(path, prefix));
1416     }
1417     return this._cdnDirPrefixes.some(prefix => path.startsWith(prefix));
1418   }
1419   private _matchPrefix(path: string, prefix: string): boolean {
1420     return path === prefix || path.startsWith(prefix);
1421   }
1422   private _prefix(path: string): string {
1423     const normalized = this._normalize(path);
1424     return normalized.endsWith('/') ? normalized : `${normalized}/`;
1425   }
1426   private _normalize(path: string): string {
1427     return path.replace(/^\/+/ , "").replace(/^minigame\/\//, "");
1428   }
1429 }
1430 export const CdnAssetService = new CdnAssetServiceClass();
1431 type EventHandler = (...args: any[]) => void;
1432 class EventBusClass {
1433   private _listeners: Map<string, EventHandler[]> = new Map();
1434   on(event: string, handler: EventHandler): void {
1435     if (!this._listeners.has(event)) {
1436       this._listeners.set(event, []);
1437     }
1438     this._listeners.get(event)!.push(handler);
1439   }
1440   off(event: string, handler: EventHandler): void {
1441     const handlers = this._listeners.get(event);
1442     if (!handlers) return;
1443     const idx = handlers.indexOf(handler);
1444     if (idx !== -1) handlers.splice(idx, 1);
1445   }
1446   once(event: string, handler: EventHandler): void {
1447     const wrapper = (...args: any[]) => {
1448       handler(...args);
1449       this.off(event, wrapper);
1450     };
```

```
1451   this.on(event, wrapper);
1452 }
1453 emit(event: string, ...args: any[]): void {
1454   const handlers = this._listeners.get(event);
1455   if (!handlers) return;
1456   [...handlers].forEach((handler) => {
1457     try {
1458       handler(...args);
1459     } catch (e) {
1460       const err = e instanceof Error
1461         ? `${e.name}: ${e.message}\n${e.stack ?? ''}`
1462         : String(e);
1463       console.error(`[EventBus] 事件 "${e}" 处理出错: ${err}`);
1464     }
1465   });
1466 }
1467 clear(event?: string): void {
1468   if (event) this._listeners.delete(event);
1469   else this._listeners.clear();
1470 }
1471 }
1472 export const EventBus = new EventBusClass();
1473 /**
1474  * 全局游戏单例。从 huahua Game.ts 精简：保留 750 宽适配、安全区、三级 renderer 降级，
1475  * 去掉 BuildingManager / 响应式 FeatureFlags。
1476  */
1477 import * as PIXI from 'pixi.js';
1478 import { ShaderSystem } from '@pixi/core';
1479 import { TweenManager } from './TweenManager';
1480 function ensureUnsafeEvalPatch(): void {
1481   if ((ShaderSystem.prototype as any).__patched) return;
1482   Object.assign(ShaderSystem.prototype, {
1483     __patched: true,
1484     systemCheck() { /* 禁用 eval 检测 */ },
1485   });
1486 }
1487 ensureUnsafeEvalPatch();
1488 class GameClass {
1489   app!: PIXI.Application;
1490   stage: PIXI.Container;
1491   ticker: PIXI.Ticker;
1492   designWidth = 750;
1493   designHeight = 1334;
1494   screenWidth = 375;
1495   screenHeight = 667;
1496   scale = 1;
1497   contentScale = 1;
1498   contentOffsetX = 0;
1499   contentOffsetY = 0;
1500   dpr = 1;
```

```
19413     fn.call(xhr);
19414     _xhrLog(id, 'callback:' + name + 'end', "");
19415 } catch (e) {
19416     _xhrLog(id, 'callback:' + name + ':throw', _safeString(e && (e.stack || e.message) || e));
19417     throw e;
19418 }
19419 }
19420 class XMLHttpRequest {
19421     constructor() {
19422         this._id = ++_xhrSeq;
19423         this.readyState = 0;
19424         this.status = 0;
19425         this.statusText = "";
19426         this.responseText = "";
19427         this.response = null;
19428         this.responseType = "";
19429         this.responseURL = "";
19430         this.withCredentials = false;
19431         this.timeout = 0;
19432         this._method = "";
19433         this._url = "";
19434         this._headers = {};
19435         this.onreadystatechange = null;
19436         this.onload = null;
19437         this.onerror = null;
19438         this.onabort = null;
19439         this.ontimeout = null;
19440         this.onprogress = null;
19441     }
19442     open(method, url) {
19443         this._method = method;
19444         this._url = url;
19445         this.readyState = 1;
19446         _xhrLog(this._id, 'open', String(method || 'GET').toUpperCase() + ' ' + _shortUrl(url));
19447     }
19448     setRequestHeader(key, value) {
19449         this._headers[key] = value;
19450         _xhrLog(this._id, 'header', _safeString(key) + '=' + _safeString(value));
19451     }
19452     getResponseHeader(key) {
19453         return this._responseHeaders ? this._responseHeaders[key.toLowerCase()] : null;
19454     }
19455     getAllResponseHeaders() {
19456         return "";
19457     }
19458     send(data) {
19459         const self = this;
19460         const responseType = this.responseType || 'text';
19461         const headers = {};
19462         for (const key in this._headers) {
```

```

19463   headers[key] = String(this._headers[key]);
19464   }
19465   const requestData = data == null || typeof data === 'string' || data instanceof ArrayBuffer
19466   ? (data || undefined)
19467   : JSON.stringify(data);
19468   const isGet = String(this._method || 'GET').toUpperCase() === 'GET';
19469   const isHttp = /^https?:\/\//i.test(this._url || '');
19470   _xhrLog(this._id, 'send', [
19471     'method=' + String(this._method || 'GET').toUpperCase(),
19472     'responseType=' + responseType,
19473     'data=' + _dataKind(requestData),
19474     'isHttp=' + isHttp,
19475     'url=' + _shortUrl(this._url),
19476   ].join(' '));
19477   if (isGet && isHttp && !requestData && platform.downloadFile && platform.getFileSystemManager) {
19478     _xhrLog(this._id, 'transport', 'downloadFile');
19479     platform.downloadFile({
19480       url: this._url,
19481       success(res) {
19482         _xhrLog(self._id, 'downloadFile:success', 'status=' + (res && res.statusCode) + ' temp=...
19483         const fs = platform.getFileSystemManager();
19484         self.status = res.statusCode || 200;
19485         self.statusText = String(self.status);
19486         self._responseHeaders = {};
19487         try {
19488           if (responseType === 'arraybuffer') {
19489             self.response = fs.readFileSync(res.tempFilePath);
19490             _xhrLog(self._id, 'readFile', 'arraybuffer response=' + _dataKind(self.response));
19491           } else {
19492             const text = fs.readFileSync(res.tempFilePath, 'utf-8');
19493             self.responseText = text;
19494             self.response = responseType === 'json' ? JSON.parse(text) : text;
19495             _xhrLog(self._id, 'readFile', 'text=' + text.length + ' response=' + _dataKind(self...
19496           }
19497         } catch (e) {
19498           self.status = 0;
19499           self.readyState = 4;
19500           _xhrLog(self._id, 'readFile:error', _safeString(e && (e.stack || e.message) || e));
19501           _invoke(self, self._id, 'onreadystatechange');
19502           _invoke(self, self._id, 'onerror');
19503           return;
19504         }
19505         self.readyState = 4;
19506         _invoke(self, self._id, 'onreadystatechange');
19507         _invoke(self, self._id, 'onload');
19508       },
19509       fail(err) {
19510         self.status = 0;
19511         self.readyState = 4;
19512         const msg = err && (err.errMsg || err.message) ? (err.errMsg || err.message) : String(e...

```



```
19513     _xhrLog(self._id, 'downloadFile:fail', msg);
19514     _invoke(self, self._id, 'onreadystatechange');
19515     _invoke(self, self._id, 'onerror');
19516     },
19517     });
19518     return;
19519     }
19520     _xhrLog(this._id, 'transport', 'request');
19521     platform.request({
19522         url: this._url,
19523         method: this._method || 'GET',
19524         header: headers,
19525         data: requestData,
19526         responseType: responseType === 'arraybuffer' ? 'arraybuffer' : 'text',
19527         dataType: responseType === 'json' ? 'json' : undefined,
19528         success(res) {
19529             _xhrLog(self._id, 'request:success', 'status=' + (res && res.statusCode) + ' data=' + _da...
19530             self.status = res.statusCode;
19531             self.statusText = res.statusCode + ";
19532             self._responseHeaders = res.header || {};
19533             if (responseType === 'arraybuffer') {
19534                 self.response = res.data;
19535             } else if (responseType === 'json') {
19536                 self.response = res.data;
19537                 self.responseText = JSON.stringify(res.data);
19538             } else {
19539                 self.responseText = res.data;
19540                 self.response = res.data;
19541             }
19542             self.readyState = 4;
19543             _invoke(self, self._id, 'onreadystatechange');
19544             _invoke(self, self._id, 'onload');
19545         },
19546         fail(err) {
19547             self.status = 0;
19548             self.readyState = 4;
19549             const msg = err && (err.errMsg || err.message) ? (err.errMsg || err.message) : String(err);
19550             _xhrLog(self._id, 'request:fail', msg);
19551             _invoke(self, self._id, 'onreadystatechange');
19552             _invoke(self, self._id, 'onerror');
19553         },
19554     });
19555     }
19556     abort() {
19557         _xhrLog(this._id, 'abort', _shortUrl(this._url));
19558         _invoke(this, this._id, 'onabort');
19559     }
19560     addEventListener(type, handler) {
19561         _xhrLog(this._id, 'addEventListener', _safeString(type));
19562         this['on' + type] = handler;
```

```
19563 }
19564 removeEventListener() {}
19565 }
19566 Object.defineProperty(XMLHttpRequest.prototype, Symbol.toStringTag, {
19567   value: 'XMLHttpRequest',
19568   configurable: true,
19569 });
19570 // 静态常量
19571 XMLHttpRequest.UNSENT = 0;
19572 XMLHttpRequest.OPENED = 1;
19573 XMLHttpRequest.HEADERS_RECEIVED = 2;
19574 XMLHttpRequest.LOADING = 3;
19575 XMLHttpRequest.DONE = 4;
19576 module.exports = XMLHttpRequest;
19577 /**
19578  * Canvas 管理模块
19579  * 第一次 createCanvas() 返回主屏 canvas（小游戏环境特性，微信和抖音一致）
19580  */
19581 const platform = require('./platform');
19582 let canvas;
19583 try {
19584   canvas = platform.createCanvas();
19585 } catch (e) {
19586   console.error('[canvas] createCanvas 失败:', e);
19587   // 兜底：空对象防止后续模块加载崩溃
19588   canvas = { width: 0, height: 0, getContext: function() { return null; } };
19589 }
19590 module.exports = { canvas };
19591 /**
19592  * document 对象模拟
19593  */
19594 const platform = require('./platform');
19595 const Image = require('./Image');
19596 const { Element } = require('./element');
19597 const _eventListeners = {};
19598 const body = new Element();
19599 body.clientWidth = 0;
19600 body.clientHeight = 0;
19601 const documentElement = new Element();
19602 const document = {
19603   body,
19604   documentElement,
19605   readyState: 'complete',
19606   createElement(tag) {
19607     tag = tag.toLowerCase();
19608     switch (tag) {
19609       case 'canvas':
19610         return platform.createCanvas();
19611       case 'img':
19612       case 'image':
```

```
19613     return platform.createImage();
19614     default:
19615         return new Element();
19616     }
19617 },
19618 createElementNS(_ns, tag) {
19619     return this.createElement(tag);
19620 },
19621 createTextNode() {
19622     return new Element();
19623 },
19624 getElementById() {
19625     return null;
19626 },
19627 getElementsByTagName(tag) {
19628     if (tag === 'canvas') {
19629         const { canvas } = require('./canvas');
19630         return [canvas];
19631     }
19632     return [];
19633 },
19634 querySelector() {
19635     return null;
19636 },
19637 querySelectorAll() {
19638     return [];
19639 },
19640 // PixiJS EventSystem 使用 elementFromPoint 来确定事件目标
19641 elementFromPoint() {
19642     const { canvas } = require('./canvas');
19643     return canvas;
19644 },
19645 addEventListener(type, handler) {
19646     if (!_eventListeners[type]) _eventListeners[type] = [];
19647     _eventListeners[type].push(handler);
19648 },
19649 removeEventListener(type, handler) {
19650     if (!_eventListeners[type]) return;
19651     const idx = _eventListeners[type].indexOf(handler);
19652     if (idx !== -1) _eventListeners[type].splice(idx, 1);
19653 },
19654 dispatchEvent(ev) {
19655     const queue = _eventListeners[ev.type];
19656     if (queue) queue.forEach(handler => handler(ev));
19657 },
19658 // PixiJS 会检查这些
19659 fonts: {
19660     add() {},
19661     delete() {},
19662     has() { return false; },
```

```
19663   forEach() {},
19664 },
19665 hidden: false,
19666 visibilityState: 'visible',
19667 };
19668 module.exports = document;
19669 /**
19670  * DOM 元素模拟
19671  * 业界已知坑: HTMLCanvasElement / HTMLImageElement 必须用 constructor 直接赋值
19672  * 不能用 class extends, 否则 instanceof 校验会失败
19673  */
19674 const platform = require('./platform');
19675 class Element {
19676   constructor() {
19677     this.childNodes = [];
19678     this.style = { cursor: null };
19679     this.clientWidth = 0;
19680     this.clientHeight = 0;
19681     this._listeners = {};
19682   }
19683   appendChild(child) {
19684     this.childNodes.push(child);
19685     return child;
19686   }
19687   removeChild(child) {
19688     const idx = this.childNodes.indexOf(child);
19689     if (idx !== -1) this.childNodes.splice(idx, 1);
19690     return child;
19691   }
19692   addEventListener(type, handler) {
19693     if (!this._listeners[type]) this._listeners[type] = [];
19694     this._listeners[type].push(handler);
19695   }
19696   removeEventListener(type, handler) {
19697     if (!this._listeners[type]) return;
19698     const idx = this._listeners[type].indexOf(handler);
19699     if (idx !== -1) this._listeners[type].splice(idx, 1);
19700   }
19701   insertBefore() {}
19702   replaceChild() {}
19703   cloneNode() { return new Element(); }
19704   setAttribute() {}
19705   getAttribute() { return null; }
19706   getBoundingClientRect() {
19707     return { x: 0, y: 0, top: 0, left: 0, width: 0, height: 0, right: 0, bottom: 0 };
19708   }
19709 }
19710 // 通过 constructor 直接赋值 (非 extends), 确保 instanceof 正确
19711 // 鸿蒙等部分设备 createCanvas/createImage 返回对象可能无 constructor, 需容错
19712 let HTMLCanvasElement, HTMLImageElement;
```

```
19713 try {
19714   const _tmpCanvas = platform.createCanvas();
19715   HTMLCanvasElement = (_tmpCanvas && _tmpCanvas.constructor) ? _tmpCanvas.constructor : Element;
19716 } catch (e) {
19717   console.warn('[element] HTMLCanvasElement 获取失败, 回退到 Element:', e);
19718   HTMLCanvasElement = Element;
19719 }
19720 try {
19721   const _tmpImage = platform.createImage();
19722   HTMLImageElement = (_tmpImage && _tmpImage.constructor) ? _tmpImage.constructor : Element;
19723 } catch (e) {
19724   console.warn('[element] HTMLImageElement 获取失败, 回退到 Element:', e);
19725   HTMLImageElement = Element;
19726 }
19727 class HTMLVideoElement extends Element {}
19728 module.exports = {
19729   Element,
19730   HTMLCanvasElement,
19731   HTMLImageElement,
19732   HTMLVideoElement,
19733 };
19734 /**
19735  * pixi-adapter 统一入口
19736  * 根据运行环境 (真机/模拟器) 将 DOM 模拟对象挂载到全局
19737  *
19738  * 关键: 真机环境中 IIFE bundle 的自由变量 (document、window 等)
19739  * 必须在 JS 引擎的全局作用域中可达, 仅挂到 GameGlobal 不够--
19740  * GameGlobal 只是跨文件共享对象, 不是全局作用域。
19741  * 因此真机需要挂到 globalThis / global 上。
19742  */
19743 console.log('[pixi-adapter] 开始加载子模块...');
19744 var platform, noop, Image, canvas, location, document, navigator, localStorage, XMLHttpRequest, r...
19745 var Element, HTMLCanvasElement, HTMLImageElement, HTMLVideoElement;
19746 try { platform = require('./platform'); console.log('[pixi-adapter] [v] platform'); } catch(e) { ...
19747 try { noop = require('./util').noop; console.log('[pixi-adapter] [v] util'); } catch(e) { console...
19748 try { Image = require('./Image'); console.log('[pixi-adapter] [v] Image'); } catch(e) { console.e...
19749 try { canvas = require('./canvas').canvas; console.log('[pixi-adapter] [v] canvas'); } catch(e) { ...
19750 try { location = require('./location'); console.log('[pixi-adapter] [v] location'); } catch(e) { ...
19751 try { document = require('./document'); console.log('[pixi-adapter] [v] document'); } catch(e) { ...
19752 try { navigator = require('./navigator'); console.log('[pixi-adapter] [v] navigator'); } catch(e)...
19753 try { localStorage = require('./localStorage'); console.log('[pixi-adapter] [v] localStorage'); ...
19754 try { XMLHttpRequest = require('./XMLHttpRequest'); console.log('[pixi-adapter] [v] XMLHttpRequest...
19755 try { registerTouchEvents = require('./TouchEvent').registerTouchEvents; console.log('[pixi-adapt...
19756 try {
19757   var _elem = require('./element');
19758   Element = _elem.Element;
19759   HTMLCanvasElement = _elem.HTMLCanvasElement;
19760   HTMLImageElement = _elem.HTMLImageElement;
19761   HTMLVideoElement = _elem.HTMLVideoElement;
19762   console.log('[pixi-adapter] [v] element');
```

```
19763 } catch(e) {
19764   console.error('[pixi-adapter] [x] element:', e);
19765   Element = function() {};
19766   HTMLCanvasElement = Element;
19767   HTMLImageElement = Element;
19768   HTMLVideoElement = Element;
19769 }
19770 console.log('[pixi-adapter] 子模块加载完成');
19771 console.log('[pixi-adapter] XHR patch version: 2026-04-28-force-global-v2');
19772 // ===== 获取真正的JS 全局对象 =====
19773 // 优先 globalThis (ES2020+) , 其次 global (Node/V8) , 最后 GameGlobal
19774 const _realGlobal = (typeof globalThis !== 'undefined' && globalThis)
19775   || (typeof global !== 'undefined' && global)
19776   || GameGlobal;
19777 function _forceInstallGlobal(name, value, targets) {
19778   for (var i = 0; i < targets.length; i++) {
19779     var target = targets[i];
19780     if (!target) continue;
19781     try {
19782       _origDefineProperty.call(Object, target, name, { value: value, configurable: true, writable: true });
19783     } catch (e) {
19784       try { target[name] = value; } catch (_) {}
19785     }
19786   }
19787 }
19788 function _logXhrInstall(label, targets) {
19789   try {
19790     console.log('[pixi-adapter] XHR install check ' + label,
19791       'real=', !(_realGlobal && _realGlobal.XMLHttpRequest === XMLHttpRequest),
19792       'window=', typeof window !== 'undefined' && window.XMLHttpRequest === XMLHttpRequest,
19793       'GameGlobal=', typeof GameGlobal !== 'undefined' && GameGlobal.XMLHttpRequest === XMLHttpRequest,
19794       'ctor=', XMLHttpRequest && XMLHttpRequest.name);
19795   } catch (e) {
19796     console.warn('[pixi-adapter] XHR install check failed:', e);
19797   }
19798 }
19799 // ===== Patch Object.defineProperty =====
19800 const _origDefineProperty = Object.defineProperty;
19801 Object.defineProperty = function safeDefineProperty(obj, prop, descriptor) {
19802   try {
19803     return _origDefineProperty.call(Object, obj, prop, descriptor);
19804   } catch (e) {
19805     if (e instanceof TypeError) return obj;
19806     throw e;
19807   }
19808 };
19809 const _origDefineProperties = Object.defineProperties;
19810 Object.defineProperties = function safeDefineProperties(obj, props) {
19811   for (const key in props) {
19812     if (Object.prototype.hasOwnProperty.call(props, key)) {
```

```
19813     try {
19814         _origDefineProperty.call(Object, obj, key, props[key]);
19815     } catch (e) {
19816         if (!(e instanceof TypeError)) throw e;
19817     }
19818 }
19819 }
19820 return obj;
19821 };
19822 // ===== 获取系统信息 =====
19823 const sysInfo = platform.getSystemInfoSync();
19824 const isDevtools = sysInfo.platform === 'devtools';
19825 // ===== 定时器 & 动画帧 polyfill =====
19826 // 真机 IIFE bundle 可能无法以自由变量访问这些 API,
19827 // 在 adapter 模块作用域中它们可用, 挂到真正的全局对象。
19828 ;(function _patchTimers() {
19829     var pairs = {};
19830     if (typeof setTimeout !== 'undefined')      pairs.setTimeout = setTimeout;
19831     if (typeof clearTimeout !== 'undefined')    pairs.clearTimeout = clearTimeout;
19832     if (typeof setInterval !== 'undefined')    pairs.setInterval = setInterval;
19833     if (typeof clearInterval !== 'undefined')  pairs.clearInterval = clearInterval;
19834     if (typeof requestAnimationFrame !== 'undefined') pairs.requestAnimationFrame = requestAnima...
19835     if (typeof cancelAnimationFrame !== 'undefined') pairs.cancelAnimationFrame = cancelAnimati...
19836     for (var k in pairs) {
19837         if (typeof _realGlobal[k] === 'undefined') _realGlobal[k] = pairs[k];
19838         if (typeof GameGlobal[k] === 'undefined') GameGlobal[k] = pairs[k];
19839     }
19840 })();
19841 // ===== 禁用 OffscreenCanvas =====
19842 if (typeof GameGlobal !== 'undefined') {
19843     GameGlobal.OffscreenCanvas = undefined;
19844     _realGlobal.OffscreenCanvas = undefined;
19845 }
19846 // ===== WebGL / Canvas2D 上下文构造函数 =====
19847 // 业界已知坑: Android/鸿蒙 WebGL contextAttributes 中 stencil 返回数字 0 而不是 false,
19848 // 导致 PixiJS 判断为不支持 stencil, Filter/Mask/Graphics 全部失效
19849 let _WebGLRenderingContext = {};
19850 try {
19851     const _tmpCanvas = platform.createCanvas();
19852     console.log('[pixi-adapter] WebGL: tmpCanvas created, type:', typeof _tmpCanvas);
19853     if (_tmpCanvas && typeof _tmpCanvas.getContext === 'function') {
19854         // 业界经验: 优先 webgl, 不要尝试 webgl2 (鸿蒙/部分安卓返回假上下文)
19855         var _tmpGl = _tmpCanvas.getContext('webgl', {
19856             stencil: true,
19857             antialias: true,
19858             alpha: true,
19859             depth: true,
19860             preserveDrawingBuffer: true,
19861         });
19862         if (_tmpGl) {
```

```
19863 _WebGLRenderingContext = _tmpGl.constructor || {};  
19864 console.log('[pixi-adapter] WebGL context 获取成功');  
19865 // 业界已知坑修复: contextAttributes 中 stencil/antialias 返回 0/1 而不是 bool  
19866 // PixiJS 用 === true 判断, 导致功能被误关闭  
19867 try {  
19868     var _origGetCtxAttr = _tmpGl.getContextAttributes;  
19869     if (_origGetCtxAttr) {  
19870         var _patchProto = Object.getPrototypeOf(_tmpGl);  
19871         if (_patchProto) {  
19872             _patchProto.getContextAttributes = function() {  
19873                 var attr = _origGetCtxAttr.call(this);  
19874                 if (attr) {  
19875                     // 强制布尔化 (修复返回 0/1 的 bug)  
19876                     attr.stencil = !!attr.stencil;  
19877                     attr.antialias = !!attr.antialias;  
19878                     attr.alpha = !!attr.alpha;  
19879                     attr.depth = !!attr.depth;  
19880                     attr.preserveDrawingBuffer = !!attr.preserveDrawingBuffer;  
19881                 }  
19882                 return attr;  
19883             };  
19884             console.log('[pixi-adapter] WebGL getContextAttributes 已 patch (布尔化)');  
19885         }  
19886     }  
19887 } catch (e3) {  
19888     console.warn('[pixi-adapter] patch getContextAttributes 失败:', e3);  
19889 }  
19890 // 业界已知坑: OES_vertex_array_object 在部分 Android/鸿蒙上返回假对象  
19891 try {  
19892     var _vaoExt = _tmpGl.getExtension('OES_vertex_array_object');  
19893     if (_vaoExt && typeof _vaoExt.createVertexArrayOES !== 'function') {  
19894         // 假扩展, 禁用它  
19895         var _origGetExt = _tmpGl.__proto__.getExtension;  
19896         _tmpGl.__proto__.getExtension = function(name) {  
19897             if (name === 'OES_vertex_array_object') return null;  
19898             return _origGetExt.call(this, name);  
19899         };  
19900         console.warn('[pixi-adapter] OES_vertex_array_object 为假扩展, 已禁用');  
19901     }  
19902 } catch (e4) { /* 忽略 */ }  
19903 } else {  
19904     console.warn('[pixi-adapter] WebGL context 获取失败 (getContext 返回 null)');  
19905 }  
19906 }  
19907 } catch (e) {  
19908     console.warn('[pixi-adapter] WebGL 初始化异常:', e);  
19909 }  
19910 let _CanvasRenderingContext2D = {};  
19911 try {  
19912     const _tmpCanvas2 = platform.createCanvas();
```



```

19913 if (_tmpCanvas2 && typeof _tmpCanvas2.getContext === 'function') {
19914     const _tmpCtx = _tmpCanvas2.getContext('2d');
19915     if (_tmpCtx) {
19916         _CanvasRenderingContext2D = _tmpCtx.constructor || {};
19917         console.log('[pixi-adapter] Canvas2D context 获取成功');
19918     } else {
19919         console.warn('[pixi-adapter] Canvas2D context 获取失败');
19920     }
19921 }
19922 } catch (e) {
19923     console.warn('[pixi-adapter] Canvas2D 初始化异常:', e);
19924 }
19925 // ===== DOMParser =====
19926 class DOMParser {
19927     parseFromString() {
19928         return { documentElement: new Element() };
19929     }
19930 }
19931 // ===== performance =====
19932 const _performance = typeof performance !== 'undefined' ? performance : {
19933     now: Date.now.bind(Date),
19934 };
19935 // ===== window 事件系统 =====
19936 const _windowListeners = {};
19937 var _winEvtLogCount = 0;
19938 function _windowAddEventListener(type, handler, options) {
19939     if (!_windowListeners[type]) _windowListeners[type] = [];
19940     _windowListeners[type].push(handler);
19941     _winEvtLogCount++;
19942     if (_winEvtLogCount <= 20) {
19943         console.log('[pixi-adapter] globalThis.addEventListener 注册:', type, '(共' + _windowListeners[t...
19944     }
19945 }
19946 function _windowRemoveEventListener(type, handler) {
19947     if (!_windowListeners[type]) return;
19948     const idx = _windowListeners[type].indexOf(handler);
19949     if (idx !== -1) _windowListeners[type].splice(idx, 1);
19950 }
19951 function _windowDispatchEvent(type, event) {
19952     const queue = _windowListeners[type];
19953     if (queue) {
19954         const copy = queue.slice();
19955         copy.forEach(handler => {
19956             try { handler(event); } catch (e) { console.error('[window event]', type, e); }
19957         });
19958     }
19959 }
19960 GameGlobal.__windowDispatchEvent = _windowDispatchEvent;
19961 // ===== 事件构造函数 =====
19962 function _PointerEvent(type, opts) { this.type = type; Object.assign(this, opts || {}); }

```

```
19963 function _TouchEventCtor(type, opts) { this.type = type; Object.assign(this, opts || {}); }
19964 function _MouseEvent(type, opts) { this.type = type; Object.assign(this, opts || {}); }
19965 // ===== URL / Blob =====
19966 // Pixi 路径解析会 `new URL(src, base)`。微信里没有 URL 构造函数；
19967 // 模拟器若用对象覆盖原生 URL，同样会炸成 "URL is not a constructor"。
19968 function MiniURL(url, base) {
19969   if (!(this instanceof MiniURL)) return new MiniURL(url, base);
19970   var href = String(url == null ? "" : url);
19971   if (base && !/^([a-zA-Z][a-zA-Z\d+\-.]*:\/\.[^\/]*\/)/.test(href)) {
19972     var b = String(base);
19973     if (href.charAt(0) === '/') {
19974       var origin = b.match(/^([a-zA-Z][a-zA-Z\d+\-.]*:\/\.[^\/]*\/)/);
19975       href = (origin ? origin[0] : "") + href;
19976     } else {
19977       var cut = b.lastIndexOf('/');
19978       href = (cut >= 0 ? b.slice(0, cut + 1) : b) + href;
19979     }
19980   }
19981   this.href = href;
19982   this.pathname = href;
19983   this.search = "";
19984   this.hash = "";
19985   this.origin = "";
19986   this.protocol = "";
19987   this.host = "";
19988   this.hostname = "";
19989   this.port = "";
19990 }
19991 MiniURL.prototype.toString = function() { return this.href; };
19992 MiniURL.createObjectURL = function() { return ""; };
19993 MiniURL.revokeObjectURL = function() {};
19994 const _nativeURL = typeof URL === 'function' ? URL : null;
19995 const _URL = _nativeURL || MiniURL;
19996 function _Blob() {}
19997 // ===== 所有需要挂载的全局属性 =====
19998 const _allGlobals = {
19999   window: null,      // 下面特殊处理
20000   document: document,
20001   navigator: navigator,
20002   location: location,
20003   Image: Image,
20004   Element: Element,
20005   HTMLCanvasElement: HTMLCanvasElement,
20006   HTMLImageElement: HTMLImageElement,
20007   HTMLVideoElement: HTMLVideoElement,
20008   WebGLRenderingContext: _WebGLRenderingContext,
20009   CanvasRenderingContext2D: _CanvasRenderingContext2D,
20010   XMLHttpRequest: XMLHttpRequest,
20011   DOMParser: DOMParser,
20012   localStorage: localStorage,
```

```
20013 performance: _performance,
20014 canvas: canvas,
20015 ontouchstart: noop,
20016 addEventListener: _windowAddEventListener,
20017 removeEventListener: _windowRemoveEventListener,
20018 self: null, // 下面特殊处理
20019 PointerEvent: _PointerEvent,
20020 TouchEvent: _TouchEventCtor,
20021 MouseEvent: _MouseEvent,
20022 URL: _URL,
20023 Blob: _Blob,
20024 };
20025 if (isDevtools) {
20026 // ===== 模拟器环境 =====
20027 // window 已存在（浏览器环境），用 defineProperty 补充/覆盖
20028 const _win = typeof window !== 'undefined' ? window : GameGlobal;
20029 const _forceDevtoolsOverwrite = ['XMLHttpRequest'];
20030 _forceInstallGlobal('XMLHttpRequest', XMLHttpRequest, [_win, _realGlobal, typeof GameGlobal !== ...
20031 _logXhrInstall('devtools-before-loop');
20032 for (const key in _allGlobals) {
20033 if (key === 'window' || key === 'self') continue;
20034 if (key === 'URL' && typeof URL === 'function') continue;
20035 try {
20036 const desc = Object.getOwnPropertyDescriptor(_win, key);
20037 const force = _forceDevtoolsOverwrite.indexOf(key) !== -1;
20038 if (force || !desc || desc.configurable) {
20039 _origDefineProperty.call(Object, _win, key, { value: _allGlobals[key], configurable: true...
20040 }
20041 } catch (e) {
20042 try {
20043 if (_forceDevtoolsOverwrite.indexOf(key) !== -1) _win[key] = _allGlobals[key];
20044 } catch (_) { /* 只读属性忽略 */ }
20045 }
20046 try {
20047 if (_forceDevtoolsOverwrite.indexOf(key) !== -1) GameGlobal[key] = _allGlobals[key];
20048 } catch (_) {}
20049 }
20050 _forceInstallGlobal('XMLHttpRequest', XMLHttpRequest, [_win, _realGlobal, typeof GameGlobal !== ...
20051 _logXhrInstall('devtools-after-loop');
20052 // 关键修复：包装 window.addEventListener / removeEventListener
20053 // PixiJS EventSystem 在 globalThis(window) 上注册 pointermove / pointerup,
20054 // 但 adapter 通过 _windowListeners 分发触摸事件--两个系统完全隔离。
20055 // 包装后 handler 会同时进入 _windowListeners, adapter 的 dispatchToWindow 就能触达 PixiJS。
20056 try {
20057 var _nativeWinAdd = _win.addEventListener.bind(_win);
20058 var _nativeWinRemove = _win.removeEventListener.bind(_win);
20059 var _adapterOnlyEvents = {
20060 pointerdown: true,
20061 pointermove: true,
20062 pointerup: true,
```

```
20063     pointercancel: true,
20064     pointerover: true,
20065     pointerout: true,
20066     pointerleave: true,
20067     touchstart: true,
20068     touchmove: true,
20069     touchend: true,
20070     touchcancel: true,
20071     wheel: true,
20072   };
20073   var _wrappedAdd = function(type, handler, options) {
20074     _windowAddEventListener(type, handler, options);
20075     if (_adapterOnlyEvents[type]) {
20076       return undefined;
20077     }
20078     return _nativeWinAdd(type, handler, options);
20079   };
20080   var _wrappedRemove = function(type, handler, options) {
20081     _windowRemoveEventListener(type, handler);
20082     if (_adapterOnlyEvents[type]) {
20083       return undefined;
20084     }
20085     return _nativeWinRemove(type, handler, options);
20086   };
20087   // 微信新版基础库 addEventListener 可能为只读，用 defineProperty 强制覆盖
20088   _origDefineProperty.call(Object, _win, 'addEventListener', {
20089     value: _wrappedAdd, configurable: true, writable: true
20090   });
20091   _origDefineProperty.call(Object, _win, 'removeEventListener', {
20092     value: _wrappedRemove, configurable: true, writable: true
20093   });
20094   console.log('[pixi-adapter] window.addEventListener 已包装');
20095   } catch (e) {
20096     console.warn('[pixi-adapter] 包装 window.addEventListener 失败:', e);
20097   }
20098   // document 属性补充
20099   try {
20100     for (const key in document) {
20101       const desc = Object.getOwnPropertyDescriptor(_win.document, key);
20102       if (!desc || !desc.configurable) {
20103         _origDefineProperty.call(Object, _win.document, key, { value: document[key], configurable: true });
20104       }
20105     }
20106   } catch (e) { /* 忽略 */ }
20107   } else {
20108     // ===== 真机环境 =====
20109     // 关键：必须同时挂载到 _realGlobal (JS 引擎全局对象) 和 GameGlobal (跨文件共享)
20110     // 这样 IIFE bundle 中的自由变量 document、window 等才能正确解析
20111     // window = 全局对象自身 (模拟浏览器行为)
20112     try { _realGlobal.window = _realGlobal; } catch (e) { /* 只读忽略 */ }
```

```
20113 try { GameGlobal.window = _realGlobal; } catch (e) { /* 只读忽略 */ }
20114 // self = 全局对象自身
20115 try { _realGlobal.self = _realGlobal; } catch (e) { /* 只读忽略 */ }
20116 try { GameGlobal.self = _realGlobal; } catch (e) { /* 只读忽略 */ }
20117 // addEventListener/removeEventListener 必须强制覆盖:
20118 // 微信框架可能内置了无效版本, PixiJS EventSystem 在 self 上注册
20119 // pointermove/pointerup 依赖这些函数正确工作
20120 var _forceOverwrite = ['addEventListener', 'removeEventListener', 'URL'];
20121 for (const key in _allGlobals) {
20122   if (key === 'window' || key === 'self') continue;
20123   var val = _allGlobals[key];
20124   var force = _forceOverwrite.indexOf(key) !== -1;
20125   // 挂到真正的全局作用域
20126   if (force || typeof _realGlobal[key] === 'undefined') {
20127     try {
20128       _origDefineProperty.call(Object, _realGlobal, key, { value: val, configurable: true, writ...
20129     } catch (e) {
20130       try { _realGlobal[key] = val; } catch (e2) { /* 忽略 */ }
20131     }
20132   }
20133   // 同时挂到 GameGlobal
20134   if (force || typeof GameGlobal[key] === 'undefined') {
20135     try {
20136       _origDefineProperty.call(Object, GameGlobal, key, { value: val, configurable: true, writ...
20137     } catch (e) {
20138       try { GameGlobal[key] = val; } catch (e2) { /* 忽略 */ }
20139     }
20140   }
20141 }
20142 _forceInstallGlobal('XMLHttpRequest', XMLHttpRequest, [_realGlobal, typeof GameGlobal !== 'unde...
20143 _logXHRInstall('device-after-loop');
20144 // 确认事件系统已正确挂载
20145 console.log('[pixi-adaptor] 真机事件系统检查:',
20146   'globalThis.addEventListener === _windowAddEventListener:', _realGlobal.addEventListener === ...
20147   ', self.addEventListener === _windowAddEventListener:', (_realGlobal.self && _realGlobal.self...
20148 }
20149 // ===== 全局 canvas =====
20150 // 微信框架可能已将 canvas 设为只读属性, 需 try-catch 保护
20151 try { GameGlobal.canvas = canvas; } catch (e) { /* 已由框架设置 */ }
20152 try { _realGlobal.canvas = canvas; } catch (e) { /* 只读属性忽略 */ }
20153 // ===== navigator.userAgent =====
20154 try {
20155   if (_realGlobal.window && _realGlobal.window.navigator) {
20156     _realGlobal.window.navigator.userAgent = navigator.userAgent;
20157   }
20158 } catch (e) { /* 只读属性忽略 */ }
20159 // ===== 注册触摸事件 =====
20160 registerTouchEvents();
20161 console.log('[pixi-adaptor] 初始化完成. 平台:', platform.name, ', 环境:', isDevtools ? '模拟器' : '真机');
20162 console.log('[pixi-adaptor] _realGlobal === GameGlobal:', _realGlobal === GameGlobal,
```

```
20163   ', typeof document:', typeof _realGlobal.document,
20164   ', typeof window:', typeof _realGlobal.window);
20165 /**
20166  * localStorage 模拟，基于平台 Storage API
20167  */
20168 const platform = require('./platform');
20169 const localStorage = {
20170   getItem(key) {
20171     try {
20172       return platform.getStorageSync(key) || null;
20173     } catch (e) {
20174       return null;
20175     }
20176   },
20177   setItem(key, value) {
20178     try {
20179       platform.setStorageSync(key, value);
20180     } catch (e) {
20181       console.warn('[localStorage] setItem failed:', key, e);
20182     }
20183   },
20184   removeItem(key) {
20185     try {
20186       platform.removeStorageSync(key);
20187     } catch (e) {
20188       console.warn('[localStorage] removeItem failed:', key, e);
20189     }
20190   },
20191   clear() {
20192     console.warn('[localStorage] clear() not implemented in minigame');
20193   },
20194   get length() {
20195     return 0;
20196   },
20197   key() {
20198     return null;
20199   },
20200 };
20201 module.exports = localStorage;
20202 /**
20203  * location 对象模拟
20204  */
20205 const location = {
20206   href: 'game.js',
20207   protocol: 'https:',
20208   host: 'minigame',
20209   hostname: 'minigame',
20210   port: '',
20211   pathname: '/game.js',
20212   search: ''
```

```
20213   hash: ",
20214   origin: 'https://minigame',
20215 };
20216 module.exports = location;
20217 /**
20218  * navigator 对象模拟
20219  */
20220 const platform = require('./platform');
20221 let _sysInfo;
20222 try {
20223   _sysInfo = platform.getSystemInfoSync();
20224 } catch (e) {
20225   _sysInfo = {};
20226 }
20227 // 业界核心修复: 鸿蒙 platform 为 'ohos', 很多引擎只判断 android/ios
20228 // 需要在 userAgent 中统一输出 Android 标识, 让 PixiJS 等框架正确识别为移动设备
20229 const _platform = _sysInfo.platform || 'unknown';
20230 const _isOHOS = _platform === 'ohos';
20231 const _system = _sysInfo.system || '';
20232 // 生成兼容 UA: 鸿蒙走 Android UA (业界通用做法), 确保引擎不误判为桌面
20233 let _userAgent;
20234 if (_isOHOS) {
20235   _userAgent = 'Mozilla/5.0 (Linux; Android 12; HarmonyOS; ' + (_sysInfo.model || 'HUAWEI') + ') ...
20236 } else if (_platform === 'android') {
20237   _userAgent = 'Mozilla/5.0 (Linux; Android; ' + (_sysInfo.model || '') + ') AppleWebKit/537.36 M...
20238 } else if (_platform === 'ios') {
20239   _userAgent = 'Mozilla/5.0 (iPhone; CPU iPhone OS) AppleWebKit/537.36 MiniGame PixiJS/7';
20240 } else {
20241   _userAgent = 'Mozilla/5.0 (MiniGame; ' + _platform + ') PixiJS/7';
20242 }
20243 console.log('[navigator] platform:', _platform, ', system:', _system, ', isOHOS:', _isOHOS);
20244 const navigator = {
20245   platform: _isOHOS ? 'Linux armv8l' : (_sysInfo.platform || 'unknown'), // 鸿蒙伪装 Linux ARM
20246   language: _sysInfo.language || 'zh_CN',
20247   appVersion: '5.0 (MiniGame)',
20248   userAgent: _userAgent,
20249   onLine: true,
20250   maxTouchPoints: 10,
20251   vendor: "",
20252   product: "",
20253   productSub: "",
20254   hardwareConcurrency: 4,
20255   // PixiJS 可能会检查 gpu 信息
20256   gpu: "",
20257 };
20258 module.exports = navigator;
20259 /**
20260  * 平台抽象层 - 统一微信/抖音小游戏 API
20261  * 所有 adapter 模块通过此模块调用平台 API, 不直接写 wx.xxx 或 tt.xxx
20262  */
```

```
20263 const _isWechat = typeof wx !== 'undefined';
20264 const _isDouyin = typeof tt !== 'undefined';
20265 const _api = _isWechat ? wx : _isDouyin ? tt : null;
20266 if (!_api) {
20267   console.error('[platform] 未检测到小游戏运行环境 (wx/tt) ');
20268 }
20269 // 安全调用包装: 防止鸿蒙等环境中 API 缺失导致崩溃
20270 function _safeCall(fn, fallback) {
20271   try {
20272     return fn();
20273   } catch (e) {
20274     console.warn('[platform] API 调用失败:', e);
20275     return fallback;
20276   }
20277 }
20278 const noop = function() {};
20279 function _safeLogValue(v) {
20280   if (v == null) return "";
20281   try { return String(v); } catch (_) { return '[unstringifiable]'; }
20282 }
20283 function _shortUrl(url) {
20284   const s = _safeLogValue(url);
20285   return s.length > 220 ? s.slice(0, 220) + '...' : s;
20286 }
20287 function _sanitizeRequestOptions(opts) {
20288   const out = {
20289     url: String(opts && opts.url || ""),
20290     method: String(opts && opts.method || 'GET'),
20291   };
20292   if (opts && opts.header) {
20293     out.header = {};
20294     for (const k in opts.header) out.header[k] = String(opts.header[k]);
20295   }
20296   if (opts && opts.data !== undefined) {
20297     out.data = typeof opts.data === 'string' || opts.data instanceof ArrayBuffer
20298       ? opts.data
20299       : JSON.stringify(opts.data);
20300   }
20301   if (opts && opts.responseType) out.responseType = String(opts.responseType);
20302   if (opts && opts.dataType) out.dataType = String(opts.dataType);
20303   if (opts && opts.timeout) out.timeout = Number(opts.timeout);
20304   if (opts && typeof opts.success === 'function') out.success = opts.success;
20305   if (opts && typeof opts.fail === 'function') {
20306     out.fail = function(err) {
20307       const msg = err && (err.errMsg || err.message) ? (err.errMsg || err.message) : String(err);
20308       opts.fail({ errMsg: msg });
20309     };
20310   }
20311   if (opts && typeof opts.complete === 'function') out.complete = opts.complete;
20312   return out;
```



```
20313 }
20314 const platform = {
20315   createCanvas: () => _api ? _api.createCanvas() : { width: 0, height: 0, getContext: function() ...
20316   createImage: () => _api ? _api.createImage() : { src: "", onload: null, onerror: null },
20317   getSystemInfoSync: () => _api ? _safeCall(() => _api.getSystemInfoSync(), { platform: 'unknown'...
20318   getWindowInfo: () => {
20319     if (_api && _api.getWindowInfo) {
20320       try {
20321         const info = _api.getWindowInfo();
20322         if (info && (info.windowWidth || info.screenWidth) && (info.windowHeight || info.screenHe...
20323           return info;
20324       }
20325     } catch (_) {}
20326   }
20327   return _api
20328     ? _safeCall(() => _api.getSystemInfoSync(), { windowWidth: 375, windowHeight: 667 })
20329     : { windowWidth: 375, windowHeight: 667 };
20330 },
20331   getStorageSync: (key) => _api ? _safeCall(() => _api.getStorageSync(key), "") : "",
20332   setStorageSync: (key, data) => _api ? _safeCall(() => _api.setStorageSync(key, data)) : undefined,
20333   removeStorageSync: (key) => _api ? _safeCall(() => _api.removeStorageSync(key)) : undefined,
20334   request: (opts) => {
20335     if (!_api) return null;
20336     const clean = _sanitizeRequestOptions(opts || {});
20337     try {
20338       console.log('[platform.request]', clean.method, _shortUrl(clean.url), 'responseType=' + (cl...
20339     } catch (_) {}
20340     return _api.request(clean);
20341   },
20342   downloadFile: (opts) => {
20343     if (!_api || !_api.downloadFile) return null;
20344     const clean = {
20345       url: String(opts && opts.url || ""),
20346     };
20347     if (opts && typeof opts.success === 'function') clean.success = opts.success;
20348     if (opts && typeof opts.fail === 'function') {
20349       clean.fail = function(err) {
20350         const msg = err && (err.errMsg || err.message) ? (err.errMsg || err.message) : String(err);
20351         opts.fail({ errMsg: msg });
20352       };
20353     }
20354     if (opts && typeof opts.complete === 'function') clean.complete = opts.complete;
20355     if (opts && opts.timeout) clean.timeout = Number(opts.timeout);
20356     try { console.log('[platform.downloadFile]', _shortUrl(clean.url)); } catch (_) {}
20357     return _api.downloadFile(clean);
20358   },
20359   getFileSystemManager: () => _api && _api.getFileSystemManager ? _api.getFileSystemManager() : n...
20360   connectSocket: (opts) => _api ? _api.connectSocket(opts) : null,
20361   onTouchStart: (cb) => _api && _api.onTouchStart ? _api.onTouchStart(cb) : noop,
20362   onTouchMove: (cb) => _api && _api.onTouchMove ? _api.onTouchMove(cb) : noop,
```

```
20363 onTouchEnd: (cb) => _api && _api.onTouchEnd ? _api.onTouchEnd(cb) : noop,
20364 onTouchCancel: (cb) => _api && _api.onTouchCancel ? _api.onTouchCancel(cb) : noop,
20365 offTouchStart: (cb) => _api && _api.offTouchStart ? _api.offTouchStart(cb) : noop,
20366 offTouchMove: (cb) => _api && _api.offTouchMove ? _api.offTouchMove(cb) : noop,
20367 offTouchEnd: (cb) => _api && _api.offTouchEnd ? _api.offTouchEnd(cb) : noop,
20368 offTouchCancel: (cb) => _api && _api.offTouchCancel ? _api.offTouchCancel(cb) : noop,
20369 createInnerAudioContext: () => _api && _api.createInnerAudioContext ? _api.createInnerAudioCont...
20370 name: _isWechat ? 'wechat' : _isDouyin ? 'douyin' : 'unknown',
20371 api: _api,
20372 };
20373 module.exports = platform;
20374 /**
20375  * 工具函数
20376  */
20377 function noop() {}
20378 module.exports = { noop };
20379 /**
20380  * 捡菜市集统一后端 (CloudBase 云函数 + HTTP 访问服务)
20381  *
20382  * 路由 (全部 POST JSON) :
20383  * /login    登录: wx/dy code2session / anon, 签发 JWT
20384  * /save/pull  拉取当前用户存档
20385  * /health    健康检查 (无鉴权)
20386  *
20387  * 环境变量 (CloudBase 控制台 -> 云函数 -> 环境变量) :
20388  * GAME_KEY      必填; 游戏代号 jiancai
20389  * JIANCAI_JWT_SECRET  必填; 签发/校验 JWT
20390  * JIANCAI_WX_APPID / JIANCAI_WX_SECRET  微信 code2session
20391  * JIANCAI_TT_APPID / JIANCAI_TT_SECRET  抖音 code2session (可选)
20392  * JIANCAI_SAVE_MAX_BYTES  可选, 默认 262144 (256KB)
20393  * JIANCAI_TOKEN_TTL_SEC   可选, 默认 604800 (7d)
20394  */
20395 const { handleLogin } = require('./lib/auth');
20396 const { handlePull, handlePush } = require('./lib/save');
20397 const { respond, parseEvent, preflight } = require('./lib/http');
20398 const ROUTES = {
20399   'GET /health': async () => ({ ok: true, ts: Date.now() }),
20400   'POST /health': async () => ({ ok: true, ts: Date.now() }),
20401   'POST /login': handleLogin,
20402   'POST /save/pull': handlePull,
20403   'POST /save/push': handlePush,
20404 };
20405 exports.main = async (event, context) => {
20406   try {
20407     if (event && event.httpMethod === 'OPTIONS') {
20408       return preflight();
20409     }
20410     const req = parseEvent(event);
20411     const key = `${req.method} ${req.path}`;
20412     const handler = ROUTES[key];
```

```
20413   if (!handler) {
20414     return respond(404, { ok: false, code: 'NOT_FOUND', error: `no route: ${key}` });
20415   }
20416   const result = await handler(req, context);
20417   if (result && typeof result === 'object' && 'statusCode' in result) {
20418     return result;
20419   }
20420   return respond(200, { ok: true, data: result });
20421 } catch (e) {
20422   const code = e && e.code ? e.code : 'INTERNAL';
20423   const status = e && e.status ? e.status : 500;
20424   const message = (e && e.message) || String(e);
20425   console.error('[jiancai-api] error:', code, message, e && e.stack);
20426   const out = { ok: false, code, error: message };
20427   if (e && e.data !== undefined) out.data = e.data;
20428   return respond(status, out);
20429 }
20430 };
20431 const jwt = require('jsonwebtoken');
20432 const { httpError } = require('./http');
20433 const {
20434   getGameKey,
20435   gameKeyUpper,
20436   getJwtSecret: readJwtSecret,
20437   getTtlSec,
20438   getPlatformCredential,
20439 } = require('./config');
20440 const SUPPORTED_PLATFORMS = new Set(['wx', 'dy', 'tap', 'anon']);
20441 function getJwtSecret() {
20442   const secret = readJwtSecret();
20443   if (!secret) {
20444     throw httpError(500, 'NO_JWT_SECRET', `${gameKeyUpper()}_JWT_SECRET 未配置`);
20445   }
20446   return secret;
20447 }
20448 async function handleLogin(req) {
20449   const body = req.body || {};
20450   const platform = String(body.platform || '').toLowerCase();
20451   if (!SUPPORTED_PLATFORMS.has(platform)) {
20452     throw httpError(400, 'BAD_PLATFORM', `unsupported platform: ${platform}`);
20453   }
20454   let platformUid = '';
20455   if (platform === 'wx') {
20456     platformUid = await wxCode2Openid(body.code);
20457   } else if (platform === 'dy') {
20458     platformUid = await ttCode2Openid(body.code);
20459   } else if (platform === 'tap') {
20460     const id = String(body.taptapUserId || '').trim();
20461     if (!id) throw httpError(400, 'NO_TAP_ID', 'taptapUserId 缺失');
20462     platformUid = id;
```

```
20463 } else if (platform === 'anon') {
20464   const id = String(body.anonId || '').trim();
20465   if (!id) throw httpError(400, 'NO_ANON_ID', 'anonId 缺失');
20466   if (!/^[A-Za-z0-9_\-]{8,128}$/.test(id)) {
20467     throw httpError(400, 'BAD_ANON_ID', 'anonId 非法');
20468   }
20469   platformUid = id;
20470 }
20471 const userId = `${platform}:${platformUid}`;
20472 const ttlSec = getTtlSec();
20473 const gameKey = getGameKey();
20474 const now = Math.floor(Date.now() / 1000);
20475 const token = jwt.sign(
20476   { sub: userId, plt: platform, gk: gameKey, iat: now },
20477   getJwtSecret(),
20478   { expiresIn: ttlSec },
20479 );
20480 return {
20481   token,
20482   userId,
20483   platform,
20484   gameKey,
20485   expiresAt: (now + ttlSec) * 1000,
20486   ttlSec,
20487 };
20488 }
20489 function requireUser(req) {
20490   const authHeader = (req.headers && req.headers.authorization) || '';
20491   const match = /^Bearer\s+(.+)/i.exec(authHeader);
20492   if (!match) throw httpError(401, 'NO_TOKEN', '缺少 Authorization: Bearer <token>');
20493   let payload;
20494   try {
20495     payload = jwt.verify(match[1].trim(), getJwtSecret());
20496   } catch (error) {
20497     throw httpError(401, 'BAD_TOKEN', error && error.message ? error.message : 'token 无效');
20498   }
20499   const userId = payload && payload.sub;
20500   if (!userId || typeof userId !== 'string' || !userId.includes(':')) {
20501     throw httpError(401, 'BAD_TOKEN', 'token sub 非法');
20502   }
20503   const currentGameKey = getGameKey();
20504   if (payload.gk && payload.gk !== currentGameKey) {
20505     throw httpError(401, 'BAD_TOKEN', `token gameKey=${payload.gk} 与当前 GAME_KEY=${currentGameKey}...`);
20506   }
20507   return { userId, platform: payload.plt || userId.split(':')[0] };
20508 }
20509 async function wxCode2Openid(code) {
20510   const appid = getPlatformCredential('wx', 'APPID');
20511   const secret = getPlatformCredential('wx', 'SECRET');
20512   if (!appid || !secret) {
```

```
20513   throw httpError(500, 'NO_WX_CFG', `${gameKeyUpper()}_WX_APPID/${gameKeyUpper()}_WX_SECRET 未配置`);
20514 }
20515 if (!code) throw httpError(400, 'NO_CODE', 'wx code 缺失');
20516 const url = `https://api.weixin.qq.com/sns/jscode2session?appid=${encodeURIComponent(appid)}&se...
20517 const data = await httpGetJson(url);
20518 if (!data || !data.openid) {
20519   throw httpError(401, 'WX_LOGIN_FAIL', `wx code2session 失败: ${JSON.stringify(data || {})}`);
20520 }
20521 return data.openid;
20522 }
20523 async function ttCode2Openid(code) {
20524   const appid = getPlatformCredential('tt', 'APPID');
20525   const secret = getPlatformCredential('tt', 'SECRET');
20526   if (!appid || !secret) {
20527     throw httpError(500, 'NO_TT_CFG', `${gameKeyUpper()}_TT_APPID/${gameKeyUpper()}_TT_SECRET 未配置`);
20528   }
20529   if (!code) throw httpError(400, 'NO_CODE', 'dy code 缺失');
20530   const data = await httpPostJson('https://developer.toutiao.com/api/apps/v2/jscode2session', { a...
20531   if (!data || data.err_no !== 0 || !data.data || !data.data.openid) {
20532     throw httpError(401, 'TT_LOGIN_FAIL', `dy code2session 失败: ${JSON.stringify(data || {})}`);
20533   }
20534   return data.data.openid;
20535 }
20536 function httpGetJson(url) {
20537   return httpRequestJson(url, 'GET');
20538 }
20539 function httpPostJson(url, body) {
20540   return httpRequestJson(url, 'POST', body);
20541 }
20542 function httpRequestJson(url, method, body) {
20543   if (typeof fetch !== 'function') {
20544     return Promise.reject(httpError(500, 'NO_FETCH', '当前 Node 运行时不支持 fetch, 请使用 Node 18+'));
20545   }
20546   const opts = { method, headers: { 'Content-Type': 'application/json' } };
20547   if (body !== undefined) {
20548     opts.body = JSON.stringify(body);
20549   }
20550   return fetch(url, opts).then(async (res) => {
20551     const text = await res.text();
20552     try {
20553       return JSON.parse(text);
20554     } catch (_) {
20555       return { _raw: text };
20556     }
20557   });
20558 }
20559 module.exports = {
20560   handleLogin,
20561   requireUser,
20562 };
```

```
20563 /**
20564  * 游戏级配置（多游戏复用模板的唯一可变点）
20565  *
20566  * GAME_KEY      游戏代号，如 'jiancai'
20567  *              决定：集合前缀 ${GAME_KEY}_xxx、JWT payload 里的 gk
20568  * {GAMEKEY_UPPER}_JWT_SECRET 当前游戏的 JWT 签名密钥
20569  * {GAMEKEY_UPPER}_TOKEN_TTL_SEC token 有效期（秒）
20570  * {GAMEKEY_UPPER}_SAVE_MAX_BYTES payload 上限（字节）
20571  * {GAMEKEY_UPPER}_WX_APPID / _WX_SECRET
20572  * {GAMEKEY_UPPER}_TT_APPID / _TT_SECRET
20573  *
20574  * 集合命名约定：
20575  * ${GAME_KEY}_playerData 微信 / 匿名
20576  * ${GAME_KEY}_tt_playerData 抖音
20577 */
20578 const DEFAULT_GAME_KEY = 'jiancai';
20579 const DEFAULT_TTL_SEC = 7 * 24 * 3600;
20580 const DEFAULT_MAX_BYTES = 256 * 1024;
20581 function getGameKey() {
20582   const v = String(process.env.GAME_KEY || '').trim().toLowerCase();
20583   if (!v) return DEFAULT_GAME_KEY;
20584   if (! /^[a-z][a-z0-9_-]{0,31}$/ .test(v)) {
20585     throw new Error('GAME_KEY 非法: "${v}"（要求小写字母开头，字母数字/下划线/连字符，长度 1~32）');
20586   }
20587   return v;
20588 }
20589 function gameKeyUpper() {
20590   return getGameKey().toUpperCase().replace(/^[A-Z0-9]/g, '_');
20591 }
20592 function readEnvPrefer(...keys) {
20593   for (const k of keys) {
20594     const v = process.env[k];
20595     if (v !== undefined && v !== null && String(v).length > 0) return String(v);
20596   }
20597   return '';
20598 }
20599 /** 后端 platform 字段 -> 集合命名空间段。微信 / 匿名不加段。 */
20600 const PLATFORM_SCOPE = {
20601   dy: 'tt',
20602 };
20603 function getPlatformScope(platform) {
20604   return PLATFORM_SCOPE[String(platform || '').toLowerCase()] || '';
20605 }
20606 /** jiancai 或 jiancai_tt */
20607 function getScopedGameKey(platform) {
20608   const scope = getPlatformScope(platform);
20609   return scope ? `${getGameKey()}_${scope}` : getGameKey();
20610 }
20611 function scopedKeyUpper(platform) {
20612   return getScopedGameKey(platform).toUpperCase().replace(/^[A-Z0-9]/g, '_');
```

```
20613 }
20614 /**
20615 * 集合名: 微信 jiancai_playerData, 抖音 jiancai_tt_playerData。
20616 * 可用 {SCOPE_GAME_KEY}_{SUFFIX}_COLLECTION 环境变量覆盖。
20617 */
20618 function getCollectionName(suffix, platform) {
20619   const normalizedSuffix = String(suffix || "").replace(/^_+/, "");
20620   const override = process.env[`${scopedKeyUpper(platform)}_${normalizedSuffix.toUpperCase()}_COL...
20621   if (override) return String(override);
20622   return `${getScopedGameKey(platform)}_${normalizedSuffix}`;
20623 }
20624 function getJwtSecret() {
20625   return readEnvPrefer(`${gameKeyUpper()}_JWT_SECRET`);
20626 }
20627 function getTtlSec() {
20628   const raw = readEnvPrefer(`${gameKeyUpper()}_TOKEN_TTL_SEC`);
20629   const v = Number(raw);
20630   return Number.isFinite(v) && v > 0 ? Math.floor(v) : DEFAULT_TTL_SEC;
20631 }
20632 function getMaxBytes() {
20633   const raw = readEnvPrefer(`${gameKeyUpper()}_SAVE_MAX_BYTES`);
20634   const v = Number(raw);
20635   return Number.isFinite(v) && v > 0 ? Math.floor(v) : DEFAULT_MAX_BYTES;
20636 }
20637 function getPlatformCredential(platform, field) {
20638   const upper = gameKeyUpper();
20639   const platformUpper = platform.toUpperCase();
20640   return readEnvPrefer(`${upper}_${platformUpper}_${field}`);
20641 }
20642 module.exports = {
20643   getGameKey,
20644   gameKeyUpper,
20645   getPlatformScope,
20646   getScopedGameKey,
20647   getCollectionName,
20648   getJwtSecret,
20649   getTtlSec,
20650   getMaxBytes,
20651   getPlatformCredential,
20652 };
20653 const tcb = require('@cloudbase/node-sdk');
20654 const { getCollectionName } = require('./config');
20655 let _app = null;
20656 function getApp() {
20657   if (_app) return _app;
20658   _app = tcb.init({
20659     env: process.env.TCB_ENV || tcb.SYMBOL_CURRENT_ENV,
20660   });
20661   return _app;
20662 }
```

```
20663 function getDb() {
20664   return getApp().database();
20665 }
20666 /** 存档主表: 按平台隔离 (wx -> jiancai_playerData, dy -> jiancai_tt_playerData) */
20667 function getCollection(platform) {
20668   return getDb().collection(getCollectionName('playerData', platform));
20669 }
20670 function collection(suffix, platform) {
20671   return getDb().collection(getCollectionName(suffix, platform));
20672 }
20673 module.exports = {
20674   getApp,
20675   getDb,
20676   getCollection,
20677   collection,
20678 };
20679 const { getGameKey } = require('./config');
20680 const CORS_HEADERS = {
20681   'Access-Control-Allow-Origin': '*',
20682   'Access-Control-Allow-Methods': 'GET,POST,OPTIONS',
20683   'Access-Control-Allow-Headers': 'Content-Type,Authorization',
20684   'Access-Control-Max-Age': '86400',
20685 };
20686 function respond(statusCode, body, extraHeaders = {}) {
20687   return {
20688     statusCode,
20689     headers: {
20690       'Content-Type': 'application/json; charset=utf-8',
20691       ...CORS_HEADERS,
20692       ...extraHeaders,
20693     },
20694     body: typeof body === 'string' ? body : JSON.stringify(body),
20695     isBase64Encoded: false,
20696   };
20697 }
20698 function preflight() {
20699   return {
20700     statusCode: 204,
20701     headers: { ...CORS_HEADERS },
20702     body: "",
20703     isBase64Encoded: false,
20704   };
20705 }
20706 function parseEvent(event) {
20707   event = event || {};
20708   if (event.httpMethod) {
20709     const method = String(event.httpMethod).toUpperCase();
20710     let rawBody = event.body || "";
20711     if (event.isBase64Encoded && rawBody) {
20712       try {
```



```
20713     rawBody = Buffer.from(rawBody, 'base64').toString('utf8');
20714   } catch (_) {}
20715 }
20716 let body = {};
20717 if (rawBody) {
20718   try {
20719     body = JSON.parse(rawBody);
20720   } catch (_) {}
20721   body = {};
20722 }
20723 }
20724 return {
20725   method,
20726   path: normalizePath(event.path || '/'),
20727   body,
20728   headers: lowercaseHeaders(event.headers || {}),
20729   query: event.queryStringParameters || {},
20730   raw: event,
20731 };
20732 }
20733 const action = String(event.action || "").replace(/^\//+, "");
20734 return {
20735   method: 'POST',
20736   path: action ? `/${action}` : '/',
20737   body: event.body || {},
20738   headers: lowercaseHeaders(event.headers || {}),
20739   query: {},
20740   raw: event,
20741 };
20742 }
20743 function normalizePath(path) {
20744   if (!path) return '/';
20745   let p = String(path);
20746   if (!p.startsWith('/')) p = `/${p}`;
20747   const apiPrefix = `${getGameKey()}-api`;
20748   p = p.replace(new RegExp(`^(?:${escapeRegExp(apiPrefix)})(?=/|$)`), "");
20749   if (p === "") p = '/';
20750   if (p.length > 1 && p.endsWith('/')) p = p.slice(0, -1);
20751   return p;
20752 }
20753 function lowercaseHeaders(headers) {
20754   const out = {};
20755   for (const key of Object.keys(headers || {})) {
20756     out[key.toLowerCase()] = headers[key];
20757   }
20758   return out;
20759 }
20760 function httpError(status, code, message) {
20761   const err = new Error(message || code);
20762   err.status = status;
```

```
20763   err.code = code;
20764   return err;
20765 }
20766 function escapeRegExp(text) {
20767   return String(text).replace(/[*+?^$}{0|[\]\\\]/g, '\\$&');
20768 }
20769 module.exports = {
20770   respond,
20771   preflight,
20772   parseEvent,
20773   httpError,
20774 };
20775 const { httpError } = require('./http');
20776 const { requireUser } = require('./auth');
20777 const { getCollection } = require('./db');
20778 const { getMaxBytes } = require('./config');
20779 async function handlePull(req) {
20780   const { userId, platform } = requireUser(req);
20781   const col = getCollection(platform);
20782   const res = await col.where({ userId }).limit(1).get();
20783   const doc = (res && Array.isArray(res.data) && res.data[0]) || null;
20784   if (!doc) {
20785     return {
20786       userId,
20787       platform,
20788       exists: false,
20789       schemaVersion: 0,
20790       updatedAt: 0,
20791       payload: {},
20792       payloadKeys: [],
20793     };
20794   }
20795   return {
20796     userId,
20797     platform,
20798     exists: true,
20799     schemaVersion: doc.schemaVersion || 0,
20800     updatedAt: doc.updatedAt || 0,
20801     payload: doc.payload || {},
20802     payloadKeys: Array.isArray(doc.payloadKeys)
20803       ? doc.payloadKeys
20804       : Object.keys(doc.payload || {}),
20805     clientFingerprint: doc.clientFingerprint || "",
20806   };
20807 }
20808 async function handlePush(req) {
20809   const { userId, platform } = requireUser(req);
20810   const body = req.body || {};
20811   const schemaVersion = Number(body.schemaVersion);
20812   const updatedAt = Number(body.updatedAt);
```

```
20813 const baseRemoteUpdatedAt = Number(body.baseRemoteUpdatedAt || 0);
20814 const payload = body.payload;
20815 const clientFingerprint = String(body.clientFingerprint || "").slice(0, 200);
20816 const force = body.force === true;
20817 if (!Number.isFinite(schemaVersion) || schemaVersion <= 0) {
20818   throw httpError(400, 'BAD_SCHEMA', 'schemaVersion 非法');
20819 }
20820 if (!Number.isFinite(updatedAt) || updatedAt <= 0) {
20821   throw httpError(400, 'BAD_UPDATED_AT', 'updatedAt 非法');
20822 }
20823 if (!Number.isFinite(baseRemoteUpdatedAt) || baseRemoteUpdatedAt < 0) {
20824   throw httpError(400, 'BAD_BASE_REMOTE_UPDATED_AT', 'baseRemoteUpdatedAt 非法');
20825 }
20826 if (!payload || typeof payload !== 'object' || Array.isArray(payload)) {
20827   throw httpError(400, 'BAD_PAYLOAD', 'payload 必须是 object');
20828 }
20829 const size = Buffer.byteLength(JSON.stringify(payload), 'utf8');
20830 const maxBytes = getMaxBytes();
20831 if (size > maxBytes) {
20832   throw httpError(413, 'PAYLOAD_TOO_LARGE', `payload 超限: ${size}B > ${maxBytes}B`);
20833 }
20834 const payloadKeys = [];
20835 for (const k of Object.keys(payload)) {
20836   const v = payload[k];
20837   if (typeof v !== 'string') {
20838     throw httpError(400, 'BAD_PAYLOAD_VALUE', `payload[${k}] 必须是字符串`);
20839   }
20840   payloadKeys.push(k);
20841 }
20842 const col = getCollection(platform);
20843 const existingRes = await col.where({ userId }).limit(1).get();
20844 const existing = (existingRes && Array.isArray(existingRes.data) && existingRes.data[0]) || null;
20845 if (existing && !force) {
20846   const prevUpdatedAt = Number(existing.updatedAt) || 0;
20847   if (updatedAt < prevUpdatedAt || baseRemoteUpdatedAt < prevUpdatedAt) {
20848     throw Object.assign(
20849       httpError(
20850         409,
20851         'STALE_UPDATE',
20852         `服务端已有更新版本 remote=${prevUpdatedAt} > local=${updatedAt}, base=${baseRemoteUpdatedAt}`,
20853       ),
20854       {
20855         data: {
20856           remote: {
20857             schemaVersion: existing.schemaVersion || 0,
20858             updatedAt: prevUpdatedAt,
20859             payload: existing.payload || {},
20860             payloadKeys: Array.isArray(existing.payloadKeys)
20861               ? existing.payloadKeys
20862               : Object.keys(existing.payload || {}),
```

```
20863     },
20864     },
20865     },
20866   );
20867   }
20868 }
20869 let mergedPayload = { ...(existing && existing.payload ? existing.payload : {}) };
20870 for (const k of Object.keys(payload)) {
20871   const v = payload[k];
20872   if (v === undefined || v === null) {
20873     delete mergedPayload[k];
20874   } else {
20875     mergedPayload[k] = v;
20876   }
20877 }
20878 const mergedPayloadKeys = Object.keys(mergedPayload);
20879 const mergedSize = Buffer.byteLength(JSON.stringify(mergedPayload), 'utf8');
20880 if (mergedSize > maxBytes) {
20881   throw httpError(413, 'PAYLOAD_TOO_LARGE', `payload 超限: ${mergedSize}B > ${maxBytes}B`);
20882 }
20883 const now = Date.now();
20884 const docData = {
20885   userId,
20886   platform,
20887   schemaVersion,
20888   updatedAt,
20889   baseRemoteUpdatedAt,
20890   clientFingerprint,
20891   payload: mergedPayload,
20892   payloadKeys: mergedPayloadKeys,
20893   lastWriteAt: now,
20894 };
20895 if (existing && existing._id) {
20896   await col.doc(existing._id).update(docData);
20897   return { userId, updatedAt, savedAt: now, mode: 'update', sizeBytes: mergedSize };
20898 }
20899 const addRes = await col.add(docData);
20900 return {
20901   userId,
20902   updatedAt,
20903   savedAt: now,
20904   mode: 'insert',
20905   sizeBytes: size,
20906   _id: addRes && (addRes.id || addRes._id),
20907 };
20908 }
20909 module.exports = {
20910   handlePull,
20911   handlePush,
20912 };
```